Generative Models

From First Principles to Frontier Research

Yogesh P

February 2026

Overview

All chapters at a glance

Ch. 0 Introduction

Three ways to think about generating images: the pixel view, the feature view, and the noise view.

Part I: Explicit Likelihood Models

Ch. 1 Autoregressive Models

From FVSBN to Transformers: fixed-length models over images and audio, and variable-length sequence models.

Contents

— Part I: Explicit Likelihood Models — **4**

Chapter 1. Autoregressive Models **5**

1.1 The Problem: Modeling Joint Distributions **5**

1.1.1 The Chain Rule 5

1.2 Structural Assumptions **6**

1.2.1 Bayesian Networks 6

1.2.2 The Markov Assumption 6

1.2.3 Why Structure Alone Is Not Enough 7

1.3 Parametric Modeling of Conditional Distributions **7**

1.4 Autoregressive Models **8**

1.5 Fixed-Length Autoregressive Models **9**

1.5.1 FVSBN 10

1.5.2 NADE: Neural Autoregressive Distribution Estimators 12

1.5.3 Autoencoders: A Necessary Digression 15

1.5.4 MADE: Masked Autoencoder for Distribution Estimation 17

1.5.5 The Fixed-Length Ceiling 21

1.5.6 Summary: Fixed-Length Models 21

1.6 Variable-Length Autoregressive Models **21**

1.6.1 Words to Vectors 21

1.6.2 RNNs: The Hidden State as Context 29

1.6.3 Attention 36

1.6.4 Transformer 44

1.6.5 Summary: Variable-Length Models 56

PART I

Explicit Likelihood Models

These models define a tractable probability distribution $p_\theta(x)$ explicitly and train by maximizing the log-likelihood of the data. The key question in each chapter is: how do we make the joint distribution over high-dimensional data both expressive and computationally feasible?

Chapters in this part:

Chapter 1	Autoregressive Models
Chapter 2	Normalizing Flows *(coming soon)*
Chapter 3	Variational Autoencoders *(coming soon)*
Chapter 4	Classical Latent Variable Models *(coming soon)*

Chapter 1

Autoregressive Models

Fully Visible Models: Fixed-Length and Variable-Length

1.1 The Problem: Modeling Joint Distributions

An image can be viewed as a collection of random variables, one per pixel. More generally, any observation can be represented as a vector $(X_1, X_2, \ldots, X_n)$.

Generative modeling asks a single question:

> *How do we model the joint distribution $p(X_1, X_2, \ldots, X_n)$?*

If we can model this distribution well, we can:

- sample new data,
- condition on partial observations or prompts,
- infer hidden structure from raw inputs.

For simplicity, we assume binary variables in this chapter. (The same ideas extend to discrete or continuous settings.)

A direct approach is to represent the joint distribution explicitly as a **lookup table**: for every possible configuration of $(X_1, \ldots, X_n)$, we store its probability.

If the variables are binary, there are 2^n possible configurations. Since probabilities must sum to one, the number of independent parameters is

$$2^n - 1.$$

Thus the parameter complexity grows exponentially with n. For high-dimensional data such as images, this representation is computationally impossible.

The central challenge of generative modeling is therefore:

> *How can we represent the joint distribution without exponential cost?*

1.1.1 The Chain Rule

A natural first step is to rewrite the joint distribution using the **probability chain rule**:

$$p(X_1, \ldots, X_n) = \prod_{i=1}^{n} p(X_i \mid X_1, \ldots, X_{i-1}).$$

This decomposition is always exact — no assumptions are introduced.

However, decomposition is not compression.

If each conditional distribution is represented as a lookup table, then for binary variables the total number of parameters becomes

$$1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1,$$

which is still exponential in n.

For example, a modest 32×32 binary image would require 2^{1024} entries — clearly infeasible.

The chain rule changes the form of the problem but not its complexity. To make progress, we must introduce additional structure.

1.2 Structural Assumptions

The exponential cost arises because each variable is conditioned on *all* previous variables. In many real systems, however, only a small subset of those dependencies may actually matter.

This motivates a first strategy:

> *Reduce complexity by restricting which variables each conditional depends on.*

1.2.1 Bayesian Networks

Definition 1.1: Bayesian Network

A **Bayesian network** is a directed graphical model in which the joint distribution factorizes as

$$p(X_1, \ldots, X_n) = \prod_{i=1}^{n} p(X_i \mid \text{Parents}(X_i)).$$

Each variable depends only on a selected subset of variables called its **parents**.

If each variable has at most k parents and variables are binary, each conditional table has 2^k entries. The total parameter count becomes

$$O(n \cdot 2^k),$$

which is linear in n when k is small.

Structural assumptions reduce complexity by limiting dependencies.

1.2.2 The Markov Assumption

We can push this idea to its extreme by assuming each variable depends only on the immediately preceding one.

Definition 1.2: First-Order Markov Assumption

The **first-order Markov assumption** states

$$X_{i+1} \perp \{X_1, \ldots, X_{i-1}\} \mid X_i.$$

The resulting joint distribution becomes

$$p(X_{1:n}) = p(X_1) \prod_{i=1}^{n-1} p(X_{i+1} \mid X_i).$$

Here each variable has only one parent, so only local transition probabilities are required.

The Markov assumption is a modeling choice: we intentionally restrict dependencies to obtain tractability.

Both Bayesian networks and Markov chains follow the same philosophy: reduce complexity by narrowing *what we condition on*.

1.2.3 Why Structure Alone Is Not Enough

Structural assumptions help, but they do not fully solve the problem.

Even when dependencies are sparse, lookup tables scale exponentially with the size of the conditioning context. If the parent set becomes large, tabular representations again become intractable.

This suggests a second, fundamentally different strategy.

1.3 Parametric Modeling of Conditional Distributions

Instead of reducing dependencies, we can change how conditionals are represented.

Lookup tables memorize a probability for every possible configuration. But we can instead *compute* probabilities using a parameterized function.

Definition 1.3: Parametric Conditional Model

A **parametric conditional model** represents conditional probabilities using a parameterized function:

$$p(X_i \mid X_{<i}) = f_{\theta_i}(X_{<i}),$$

where the function outputs a valid probability distribution. The parameters may be independent for each conditional or shared across conditionals.

The key difference is conceptual:

- Lookup tables memorize each context independently.
- Functions share parameters and generalize across contexts.

Complexity is now controlled by the function class rather than by the number of possible

inputs.

This allows us to keep the full chain-rule factorization while avoiding exponential storage.

Generative models become tractable through two complementary design dimensions.

1. Structural assumptions simplify the dependency structure by restricting which variables each conditional distribution depends on:

Approach	Dependency	Parameters
Raw chain rule	All previous variables	$O(2^n)$
Bayesian network	Small parent set (k parents)	$O(n \cdot 2^k)$
Markov chain	Previous variable only	$O(n)$

2. Parametric modeling simplifies representation by replacing explicit probability tables with parameterized functions:

- Tabular models store probabilities independently for each context.
- Parametric models compute probabilities using shared parameters.

Structural assumptions control *which dependencies are modeled*, while parametric modeling controls *how conditional distributions are represented*.

These two dimensions are complementary rather than mutually exclusive. Modern generative models often combine both: structural assumptions define the factorization of the joint distribution, while parametric function approximation makes high-dimensional conditionals tractable.

1.4 Autoregressive Models

The name **autoregressive** originates from classical time-series analysis and is composed of two parts:

- **Regression** refers to predicting a variable from other variables. In statistics, regression simply means modeling a quantity as a function of observed inputs.
- **Auto** means *self*. In an autoregressive model, the inputs used for prediction come from the same sequence being modeled.

Combining the two ideas, an autoregressive model performs *self-regression*: each variable is predicted from previously observed variables in the same sequence. Conceptually, the model learns how the past explains the future.

Formally, this idea is expressed by modeling each element x_i conditioned on the preceding elements $x_{<i}$.

Definition 1.4: Autoregressive Model

An **autoregressive model** applies the chain rule factorization exactly:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1})$$

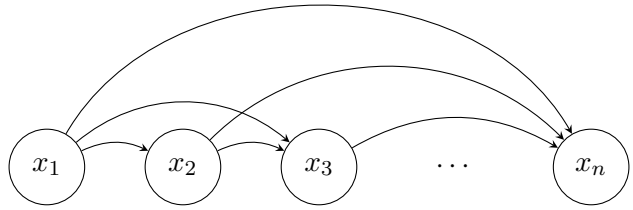

Figure 1.1: Autoregressive model with full past dependencies

but represents each conditional with a **learned function** f_{θ_i} rather than a lookup table. No independence assumptions are introduced — the full dependency structure of the chain rule is preserved.

Sampling proceeds sequentially:

1. Sample $x_1 \sim p(x_1)$
2. Sample $x_2 \sim f_{\theta_2}(x_1)$
3. Sample $x_3 \sim f_{\theta_3}(x_1, x_2)$
4. Continue until x_n

By choosing a specific function class, we implicitly restrict which distributions the model can represent. But this is a far more flexible restriction than hard independence assumptions — and it scales gracefully as the function class becomes more expressive.

Example: Autoregressive model

Consider a sequence of binary variables x_1, x_2, x_3 with learned conditionals:

$$p(x_1 = 1) = 0.6$$
$$p(x_2 = 1 \mid x_1) = f_{\theta_2}(x_1)$$
$$p(x_3 = 1 \mid x_1, x_2) = f_{\theta_3}(x_1, x_2)$$

To sample a sequence:

1. Draw x_1: get $x_1 = 1$ with probability 0.6
2. Feed x_1 into f_{θ_2} to obtain $p(x_2 = 1 \mid x_1)$, then sample x_2
3. Feed (x_1, x_2) into f_{θ_3} to obtain $p(x_3 = 1 \mid x_1, x_2)$, then sample x_3

At no point do we enumerate all $2^3 = 8$ configurations. The function does the work.

1.5 Fixed-Length Autoregressive Models

We said that instead of storing $p(x_i \mid x_1, \ldots, x_{i-1})$ as a lookup table, we could *compute* it using a learned function f_θ. But we never said what f_{θ_i} actually looks like.

This chapter builds up the answer — starting from the simplest possible parameterization

and progressively replacing its weaknesses with better ideas.

Section 1.5 covers fixed-length models. Throughout, we use MNIST as our running example: $28 \times 28 = 784$ binary pixels in raster scan order, $x_1, \ldots, x_{784} \in \{0, 1\}$.

1.5.1 FVSBN

A common and conceptually simple choice is to model each conditional distribution using logistic regression. In this setting, the conditional probability is parameterized as

$$p_\theta(x_i = 1 \mid x_{<i}) = \sigma(W_i x_{<i} + b_i)\,,$$

where $\sigma(\cdot)$ denotes the logistic sigmoid function. so each variable is predicted using a sigmoid applied to a linear function of the previous variables.. The name **Fully Visible Sigmoid Belief Network (FVSBN)** reflects three properties: the model defines probability directly over the observed variables (fully visible), each conditional distribution is parameterized with a sigmoid (logistic regression), and the joint distribution is factorized as a directed probabilistic model — historically called a belief network.

For a conditional depending on m binary variables, a lookup table requires $2^m - 1$ independent parameters. If the conditional is instead represented by a parameterized function, such as a logistic function, the number of parameters grows only linearly with m. For example, a linear logistic parameterization requires $m + 1$ parameters. The reduction in complexity comes from restricting the space of possible conditional distributions to those expressible by the chosen function.

images/chapter2_images/fvsbn.png

Figure 1.2: Illustration of FVSBN architecture

Definition 1.4: Fully Visible Sigmoid Belief Network (FVSBN)

A **Fully Visible Sigmoid Belief Network** parameterizes each conditional as:

$$\hat{x}_i = p(x_i = 1 \mid x_{<i};\, \alpha^i) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i\, x_j\right)$$

where $\sigma(z) = 1/(1+e^{-z})$ and α^i are learned parameters for position i. The joint:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_{<i})$$

Parameters: $1 + 2 + \cdots + n = O(n^2/2)$.

Example: Evaluating and Sampling from FVSBN

Define the conditional probabilities using the parameter matrix

$$\hat{x}_i = p(x_i = 1 \mid x_{<i};\, \alpha^i) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i\, x_j\right).$$

Collect the parameters into a lower–triangular matrix

$$A = \begin{bmatrix} \alpha_0^1 & 0 & 0 & 0 \\ \alpha_0^2 & \alpha_1^2 & 0 & 0 \\ \alpha_0^3 & \alpha_1^3 & \alpha_2^3 & 0 \\ \alpha_0^4 & \alpha_1^4 & \alpha_2^4 & \alpha_3^4 \end{bmatrix}, \qquad \tilde{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Then the vector of conditional probabilities can be written compactly as

$$\hat{\mathbf{x}} = \sigma(A\tilde{x}),$$

where the sigmoid function is applied element-wise.
To evaluate $p(X_1 = 0,\, X_2 = 1,\, X_3 = 1,\, X_4 = 0)$:

1. Compute all $\hat{x}_i$ using the matrix form above.
2. Multiply: $(1 - \hat{x}_1) \times \hat{x}_2 \times \hat{x}_3 \times (1 - \hat{x}_4)$

To sample:

1. Sample $x_1 \sim \text{Bernoulli}(\hat{x}_1)$
2. Sample $x_2 \sim \text{Bernoulli}(\hat{x}_2(x_1))$, and so on

The asymmetry of autoregressive models: Autoregressive models separate evaluation and generation in an important way. Given a known datapoint, the likelihood $p(x)$ can often be computed efficiently, since the conditional terms $p(x_i \mid x_{<i})$ are evaluated with inputs that

are already known. In contrast, sampling is inherently sequential: each variable must be generated before later conditionals can be evaluated. This difference between efficient likelihood evaluation and sequential generation is a defining property of autoregressive modeling.

images/chapter2_images/fvsbn2.png

Figure 1.3: Conditionals as logistic regression

Let the conditional distribution be parameterized by a function

$$f_{\theta_i}(x_{<i}) = \sigma\left(\alpha_0^i + \sum_{j=1}^{i-1} \alpha_j^i \, x_j\right),$$

which is simply a logistic regression model for variable x_i.
Then each conditional can be written compactly as

$$p(x_i = 1 \mid x_{<i}) = f_{\theta_i}(x_{<i}).$$

Using the autoregressive factorization, the joint distribution becomes

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_{<i}) = \prod_{i=1}^{n} f_{\theta_i}(x_{<i}).$$

FVSBN is valid but limited, since the function f_θ is a logistic regression and therefore models only linear dependencies. To obtain richer, more expressive conditionals, we next replace f_θ with neural networks.

1.5.2 NADE: Neural Autoregressive Distribution Estimators

NADE strengthens autoregressive modeling by replacing the linear conditional predictor of FVSBN with a one-hidden-layer neural network. For each position i, a hidden representation is computed and then mapped to a conditional probability:

$$h_i = \sigma\big(A^i x_{<i} + c\big), \qquad \hat{x}_i = \sigma\big(\alpha^i h_i + b_i\big).$$

The hidden layer allows the model to capture nonlinear interactions among observed variables, something a purely linear predictor cannot do.

The parameter-scaling problem. Assigning a separate weight matrix $A^i \in \mathbb{R}^{d\times(i-1)}$ to every conditional is costly. Summing over all positions gives

$$\sum_{i=1}^{n} d(i-1) = d\,\frac{n(n-1)}{2} = O(n^2 d)$$

parameters for the input-to-hidden mappings alone. Beyond the raw count, this design is statistically wasteful: the inputs to adjacent conditionals differ by only a single observation, yet each position learns entirely independent features. There is no mechanism for the model to transfer what it learns about earlier variables to later ones.

Weight sharing as the key idea. NADE resolves this by tying the input-to-hidden weights across all positions. A single shared matrix

$$W \in \mathbb{R}^{d\times n}$$

replaces the family $\{A^i\}$, and the i-th conditional uses only the first $i-1$ columns:

$$h_i = \sigma(W_{:,<i}\, x_{<i} + c)\,, \qquad \hat{x}_i = p(x_i = 1 \mid x_{<i}) = \sigma\big(\alpha^i h_i + b_i\big)\,.$$

Because the same column $W_{:,j}$ appears in every conditional that conditions on x_j, the model is forced to learn features that are useful across the entire sequence, not just at a single position. The input-to-hidden parameter count drops to $dn = O(nd)$, and the shared structure enables an efficient left-to-right recurrence: the pre-activation at position $i+1$ is obtained by augmenting the pre-activation at position i with a single rank-one update, rather than recomputing from scratch.

Definition 1.5: Neural Autoregressive Distribution Estimator (NADE)

NADE learns a single shared weight matrix $W \in \mathbb{R}^{d\times n}$ so that every conditional reuses the same input-to-hidden mapping:

$$h_i = \sigma(W_{:,<i}\, x_{<i} + c)\,, \qquad \hat{x}_i = p(x_i = 1 \mid x_{<i}) = \sigma\big(\alpha^i h_i + b_i\big)\,.$$

Columns of W are shared across positions, so hidden pre-activations can be updated incrementally from left to right. Compared with unshared FVSBN-style networks, NADE reduces input-to-hidden parameter count from $O(n^2 d)$ to $O(nd)$ while retaining nonlinear expressiveness.

Extensions. The NADE framework extends naturally beyond binary variables by modifying the output distribution while retaining the same autoregressive hidden representation.

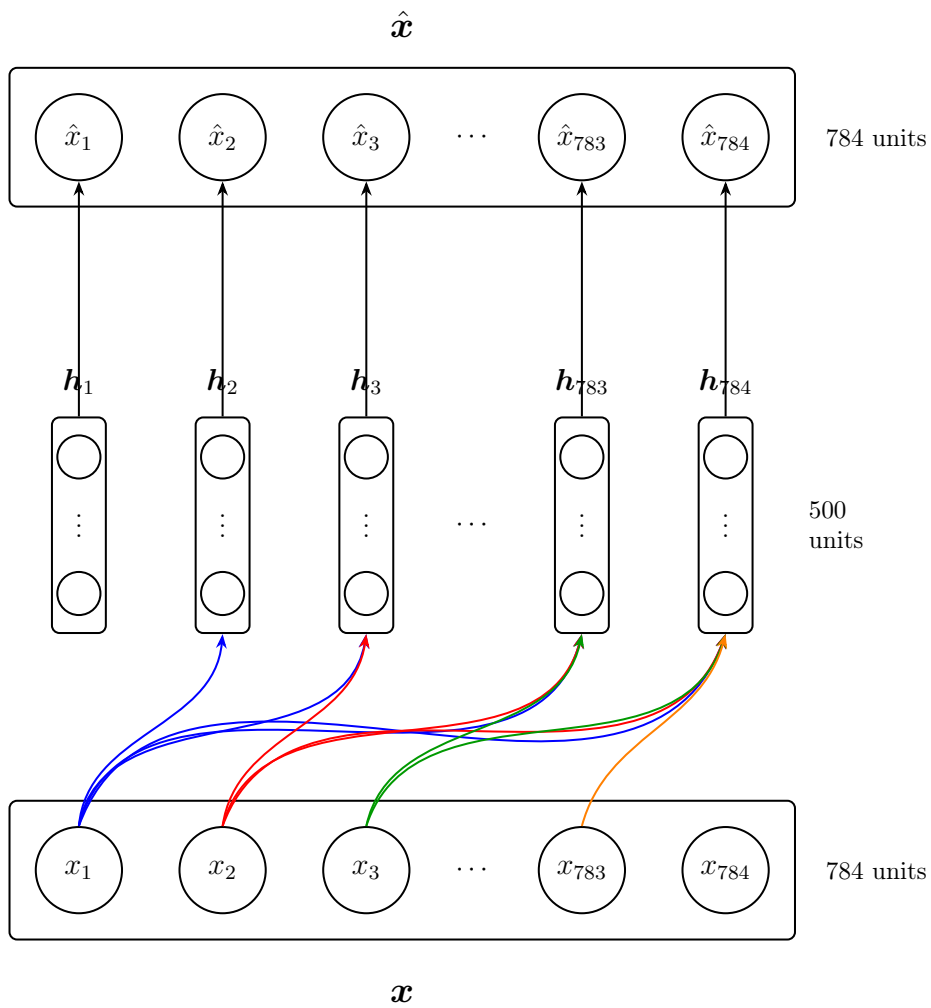

Figure 1.4: Neural Autoregressive Distribution Estimation (NADE). The input $\boldsymbol{x}$ is a 784-dimensional binary vector. Each hidden layer $\boldsymbol{h}_d$ receives only the inputs $x_1, \ldots, x_{d-1}$ (autoregressive masking), and produces the output $\hat{x}_d$ estimating the conditional probability $p(x_d = 1 \mid \boldsymbol{x}_{<d})$.

Discrete variables. For non-binary discrete random variables $x_i \in \{1, \ldots, K\}$ (e.g., pixel intensities ranging from 0 to 255), the output $\hat{x}_i$ parameterizes a categorical distribution rather than a Bernoulli. The hidden representation is computed as

$$h_i = \sigma(W_{\cdot,<i} x_{<i} + c),$$

and the conditional distribution is modeled as

$$p(x_i \mid x_{<i}) = \mathrm{Cat}(p_i^1, \ldots, p_i^K),$$

where

$$\hat{x}_i = (p_i^1, \ldots, p_i^K) = \mathrm{softmax}(A_i h_i + b_i).$$

The softmax function generalizes the sigmoid by transforming a vector of real-valued scores into a valid probability distribution over K outcomes, ensuring non-negativity and unit normalization.

Continuous variables (RNADE). For real-valued variables $x_i \in \mathbb{R}$ (e.g., speech signals), each conditional can be modeled using a continuous distribution. A common choice is a mixture of Gaussians, giving rise to the Real-valued NADE (RNADE). In this case, the conditional distribution takes the form

$$p(x_i \mid x_{<i}) = \sum_{j=1}^{K} \pi_i^j \, \mathcal{N}\Big(x_i; \mu_i^j, \sigma_i^j\Big),$$

where π_i^j are mixture weights and (μ_i^j, σ_i^j) are the mean and standard deviation of the j-th Gaussian component. These parameters are predicted from the hidden state:

$$h_i = \sigma(W_{\cdot,<i} x_{<i} + c), \qquad \hat{x}_i = (\pi_i^1, \ldots, \pi_i^K, \mu_i^1, \ldots, \mu_i^K, \sigma_i^1, \ldots, \sigma_i^K) = f(h_i).$$

In practice, the standard deviations are typically parameterized using an exponential transform to ensure $\sigma_i^j > 0$. This extension allows NADE to model complex continuous densities while preserving the autoregressive factorization and efficient shared-parameter structure.

Remark: Hidden Units as Deterministic Features Although h_i is commonly called a *hidden layer*, it is more precise to regard it as a **deterministic feature vector** derived from the observations. Given $x_{<i}$, the value of h_i is fixed:

$$h_i = \sigma(W_{:,<i}\, x_{<i} + c)\,.$$

NADE therefore contains no latent random variables. The h_i serve purely as nonlinear summaries of the observed context, used to parameterize each conditional distribution.

1.5.3 Autoencoders: A Necessary Digression

Before MADE, we need to understand autoencoders — and why they cannot be used as generative models without modification.

Definition 1.6: Autoencoder

An **autoencoder** maps input x through an encoder f_{enc} to a representation z, then through a decoder f_{dec} to a reconstruction $\hat{x}$:

$$z = f_{\text{enc}}(x), \qquad \hat{x} = f_{\text{dec}}(z), \qquad \mathcal{L} = \sum_{x \in \mathcal{D}} \sum_{i} [-x_i \log \hat{x}_i - (1 - x_i) \log(1 - \hat{x}_i)]$$

The bottleneck at z forces the network to learn a compressed representation. But a vanilla autoencoder is **not** a generative model: it does not define a distribution over x we can sample from.

images/chapter2_images/autoencoder.png

Figure 1.5: Autoencoder

On the surface, FVSBN and NADE already look like autoencoders: they take x as input and produce $\hat{x}$ with the same cross-entropy loss. The problem is that a standard autoencoder lets output $\hat{x}_i$ see input x_i directly — it can simply copy it, achieving zero reconstruction error while learning nothing useful.

The cheating problem: Output $\hat{x}_i$ must depend *only* on $x_1, \ldots, x_{i-1}$ to be useful as a generative model. At generation time we do not have x_i to copy — that is what we are trying to produce. A standard autoencoder does not enforce this constraint.

1.5.4 MADE: Masked Autoencoder for Distribution Estimation

An ordinary feed-forward autoencoder can be transformed into an autoregressive density estimator by masking its weight matrices. The goal is to preserve the autoregressive factorization

$$p(x) = \prod_{i=1}^{n} p(x_i \mid x_{<i}),$$

while computing all conditionals in a **single forward pass**.

Challenge. A standard autoencoder is fully connected, meaning each output $\hat{x}_i$ depends on all inputs. This violates the autoregressive constraint, which requires $\hat{x}_i$ to depend only on variables preceding x_i in a chosen ordering.

Key idea: masking as a structural constraint. MADE enforces a directed acyclic graph (DAG) inside the autoencoder by introducing binary masks on the weight matrices. These masks remove invalid connections so that each output obeys autoregressive dependencies. Given an ordering (or permutation) π of the variables, the joint distribution factorizes as

$$p(x) = \prod_{i=1}^{n} p(x_{\pi_i} \mid x_{\pi_{<i}}).$$

For example, if $\pi = (x_2, x_3, x_1)$, then

$$p(x_1, x_2, x_3) = p(x_2)\, p(x_3 \mid x_2)\, p(x_1 \mid x_2, x_3).$$

The output unit producing $\hat{x}_2$ is therefore not allowed to depend on any input, while the unit producing $p(x_3 \mid x_2)$ may depend only on x_2, and so forth.

Hidden-unit degrees. Instead of manually designing masks, MADE assigns each hidden unit k an integer degree

$$m(k) \in \{1, \ldots, n-1\}.$$

The degree specifies the largest input index that the hidden unit is allowed to depend on. Intuitively, it determines how far into the input ordering the unit can "see".

Mask construction. Masks are constructed using these degrees:

- Input $\to$ hidden: a hidden unit with degree $m(k)$ connects only to inputs with index $j \leq m(k)$.
- Hidden $\to$ hidden: connections satisfy

  $$m(\text{prev}) \leq m(\text{next}),$$

  ensuring autoregressive consistency across layers.

images/chapter2_images/made.png

Figure 1.6: Applying masks to Autoencoder to get MADE. Connections in light gray correspond to paths that depend only on 1 input, while the dark gray connections depend on 2 inputs.

- Hidden $\to$ output: output $\hat{x}_i$ receives connections only from hidden units with degree

$$m < i.$$

These constraints guarantee that

$$\hat{x}_i \text{ depends only on } x_{<i},$$

so the network represents a valid autoregressive model.

Here is the paragraph you can add to introduce the masked forward pass equations:

Masked forward pass. Once the degrees are assigned and the masks are constructed, the autoregressive constraints are baked directly into the network weights via the **Hadamard product** (element-wise multiplication, denoted $\odot$). For a single hidden layer, the masked forward pass is written as

$$h(x) = g\big(b + (W \odot M^W)\, x\big)\,,$$
$$\hat{x} = \sigma\big(c + (V \odot M^V)\, h(x)\big)\,,$$

where g is a nonlinear activation function, σ denotes the sigmoid, b and c are bias vectors, and W, V are the weight matrices for the input-to-hidden and hidden-to-output layers respectively. The binary matrices M^W and M^V are the masks derived from the degree assignments

described above: an entry of $M^W_{k,j} = 1$ if and only if $m(k) \geq j$, and an entry of $M^V_{i,k} = 1$ if and only if $m(k) < i$. By zeroing out the corresponding entries in the weight matrices, the Hadamard product enforces the connectivity constraints structurally, so that no gradient or information can flow through a forbidden path. The entire responsibility of satisfying the autoregressive property therefore rests with M^W and M^V, and the remaining parameters W, V, b, c are free to be optimized by standard backpropagation without any additional constraints.

Computational structure. NADE and MADE both exploit shared computations, but differ in how the autoregressive constraint is enforced. NADE computes conditionals through a left-to-right recurrence that incrementally updates hidden pre-activations across positions. MADE instead encodes the same autoregressive dependencies directly in the network connectivity via masks, allowing all outputs to be produced by a single masked forward pass without an explicit sequential recurrence.

Order-agnostic training. Autoregressive models usually assume a fixed ordering of variables (e.g., raster scan for images). This introduces an artificial bias, since variables appearing early in the order cannot depend on later ones.

MADE removes this limitation by randomly changing the variable ordering during training and regenerating the corresponding masks. Each ordering defines a different valid autoregressive factorization of the same joint distribution.

Training across many orderings reduces reliance on any single ordering and encourages the model to learn more robust dependencies. This can be viewed as implicitly training an ensemble of autoregressive models while sharing the same parameters.

An additional benefit is flexibility: by choosing an appropriate ordering, inference tasks such as imputing missing values become easier.

At sampling time, however, a single ordering is chosen, and variables are generated sequentially according to that order.

Definition 1.7: Masked Autoencoder for Distribution Estimation (MADE)

MADE is an autoregressive model obtained by applying binary masks to the weight matrices of an autoencoder. Hidden units are assigned ordering degrees that determine allowed connections, ensuring that each output $\hat{x}_i$ depends only on preceding variables. This allows all conditional distributions to be computed in a single forward pass while preserving autoregressive structure.

Example: MADE with ordering $x_2 \to x_3 \to x_1$

Consider a binary vector

$$x = (x_1, x_2, x_3), \qquad x_i \in \{0, 1\},$$

and assume the autoregressive ordering

$$x_2 \to x_3 \to x_1.$$

Autoregressive factorization. The joint distribution becomes

$$p(x) = p(x_2)\, p(x_3 \mid x_2)\, p(x_1 \mid x_2, x_3).$$

Training (stochastic gradient descent). Given a training example (x_1, x_2, x_3), MADE receives the full binary vector as input. Due to masking, the outputs represent the conditionals:

$$\hat{x}_2 = p(x_2), \qquad \hat{x}_3 = p(x_3 \mid x_2), \qquad \hat{x}_1 = p(x_1 \mid x_2, x_3).$$

Each output is a Bernoulli probability rather than a reconstructed binary value. Parameters are updated using stochastic gradient descent by minimizing the negative log-likelihood (binary cross-entropy).

Sampling (generation). After training, sampling follows the chosen ordering, requiring one forward pass per variable. We trace through each pass below.

Forward Pass 1. The input is zero-initialised since nothing has been sampled yet:

$$\text{input} = [x_1, x_2, x_3] = [0,\ 0,\ 0].$$

MADE runs forward. By construction of the masks, the output $\hat{x}_2$ has no dependencies. We read off $p(x_2)$ and sample

$$x_2 \sim \text{Bernoulli}(p(x_2)), \qquad \text{e.g. } x_2 = 1.$$

Forward Pass 2. The sampled value of x_2 is inserted into the input:

$$\text{input} = [0,\ 1,\ 0].$$

MADE runs forward again. The masks now allow $\hat{x}_3$ to attend to x_2. We read off $p(x_3 \mid x_2 = 1)$ and sample

$$x_3 \sim \text{Bernoulli}(p(x_3 \mid x_2 = 1)), \qquad \text{e.g. } x_3 = 0.$$

Forward Pass 3. Both sampled values are inserted into the input:

$$\text{input} = [0,\ 1,\ 0].$$

MADE runs forward a final time. The masks allow $\hat{x}_1$ to attend to both x_2 and x_3. We read off $p(x_1 \mid x_2 = 1,\ x_3 = 0)$ and sample

$$x_1 \sim \text{Bernoulli}(p(x_1 \mid x_2 = 1,\ x_3 = 0)), \qquad \text{e.g. } x_1 = 1.$$

The final sampled tuple is

$$(x_1, x_2, x_3) = (1, 1, 0),$$

which forms a new generated example. Although MADE computes all conditionals in one forward pass during training, generation requires D forward passes (one per variable) because each newly sampled variable changes the conditioning context. Only the output corresponding to the next variable in the ordering is read at each step; the remaining outputs are discarded.

1.5.5 The Fixed-Length Ceiling

Every model in the previous section was designed for data with a **fixed, known dimensionality**. A 28×28 image always contains exactly 784 pixels. The model is therefore constructed with exactly n variables: the masks are built for n inputs, and the network produces n conditional outputs.

This design works well for images and audio segments, where the dimensionality is known in advance. Language, however, breaks this assumption. A sentence may contain 3 words or 3000. A paragraph, a document, or an entire codebase has no fixed length known at design time. If the number of variables is not known ahead of time, we cannot build an architecture that allocates one conditional distribution for each of them.

Handling such data therefore requires a different architectural idea: instead of constructing n separate conditional models, we need a single conditional model that can be applied repeatedly as the sequence grows.

1.5.6 Summary: Fixed-Length Models

Model	Key idea	Parameters	Parallel training
FVSBN	Logistic regression per position	$O(n^2)$	✓
NADE	Shared weights, n passes	$O(nd)$	✓
MADE	Masking, single pass	$O(nd)$	✓

1.6 Variable-Length Autoregressive Models

Machine Translation: From Images to Language Language introduces two challenges that did not arise with images.

First, sequences have variable length. A sentence may contain three words or three thousand, and the model must handle both cases with the same parameters.

Second, language is symbolic rather than numerical. Pixels already live in a numeric space: each pixel value is an integer in $\{0, \ldots, 255\}$. Words, however, are discrete symbols without any inherent numerical representation.

Before we can model sequences of words, we must therefore answer a more fundamental question:

> *How do we represent a word as a mathematical object?*

1.6.1 Words to Vectors

The Naive Approach and Why It Fails

The most obvious answer to "how do we represent a word as a mathematical object" is to assign each word a unique integer. Fix a vocabulary $\mathcal{V}$ of size V — the set of all distinct words the model will ever encounter — and map each word to an index:

$$\text{coffee} \mapsto 1, \quad \text{tea} \mapsto 2, \quad \text{the} \mapsto 3, \quad \dots$$

This is compact and unambiguous, but it smuggles in a false assumption: that numbers are ordered. The model would be forced to treat "tea" as *between* "coffee" and "the" in some meaningful sense. It is not. Any arithmetic on these integers would be nonsense.

Second attempt: one-hot vectors. Instead of a single integer, give each word a vector of length V that is all zeros except for a single 1 at the position corresponding to that word:

$$\mathbf{e}_{\text{coffee}} = (1, 0, 0, \dots), \quad \mathbf{e}_{\text{tea}} = (0, 1, 0, \dots), \quad \mathbf{e}_{\text{the}} = (0, 0, 1, \dots).$$

Formally, for a word w with vocabulary index $i(w)$:

$$(\mathbf{e}_w)_j = \begin{cases} 1 & j = i(w) \\ 0 & \text{otherwise.} \end{cases}$$

This removes the false ordering. But it creates a new problem.

One-hot vectors have no geometry.

Every pair of distinct words is exactly equidistant:

$$\|\mathbf{e}_w - \mathbf{e}_{w'}\| = \sqrt{2} \qquad \text{for all } w \neq w'.$$

Equivalently, every dot product between distinct words is zero:

$$\mathbf{e}_{\text{coffee}}^\top \mathbf{e}_{\text{tea}} \;=\; \mathbf{e}_{\text{coffee}}^\top \mathbf{e}_{\text{the}} \;=\; 0.$$

"Coffee" is exactly as far from "tea" as it is from "the." The representation contains no information about which words are related.

This means there is no generalization. A model learning that "coffee" is a drink cannot apply that knowledge to "tea" — from the representation's perspective, they are unrelated objects.

These are not minor inconveniences. They mean that any model operating on one-hot vectors must learn the behavior of every word from scratch, with zero transfer between related words. For a vocabulary of hundreds of thousands of words, this is a fundamental barrier.

What We Actually Need

The failure of one-hot vectors is precise: they treat every word as equally dissimilar. What we need is a representation where **similar words are close together** in some geometric space. But close according to what?

We need a notion of word similarity that can be extracted from data, without anyone hand-labeling which words are related. The key insight comes from an observation in linguistics:

> **The distributional hypothesis:** words that appear in similar contexts tend to have similar meanings.

Consider two words you have never seen before: *grutt* and *blarpen*. You encounter them in the following sentences:

"I drank a hot *grutt* this morning."
"She ordered a *blarpen* at the café."
"The *grutt* was bitter and dark."
"He prefers *blarpen* to tea."

You do not know what these words mean, but you know they mean something similar — because they appear near the same words: *drank, hot, morning, café, bitter, tea.* The context is the meaning.

This gives us a concrete strategy: represent each word by the *company it keeps.*

Count Vectors and Their Limits

The most direct implementation: scan a large corpus and count how often each word appears near every other word. Define a *co-occurrence matrix* $C \in \mathbb{R}^{V \times V}$ where C_{ij} records how often word j appears within a fixed window of word i.

The row $C_{i,:}$ then describes the full distribution of linguistic contexts for word i — a vector representation that embodies the distributional hypothesis directly. Words with similar rows have similar contexts, and therefore similar meanings.

This works. In practice, words like "coffee" and "tea" do end up with similar rows. But it creates two practical problems.

Two failures of count vectors.

Scale. With $V = 100{,}000$ words, C has 10^{10} entries. Storing and manipulating this matrix is expensive. Most entries are zero — the matrix is extremely sparse.

Rigidity. The matrix must be recomputed from scratch whenever new text is available. There is no way to update it incrementally.

The standard fix for the scale problem is **singular value decomposition (SVD)**: approximate C by a low-rank matrix, keeping only the d most important dimensions. This compresses each row from a V-dimensional sparse vector into a dense d-dimensional vector, with $d \in \{50, 100, 300\}$. The result is a compact, dense representation for each word that still captures co-occurrence structure.

This works, but it remains a two-stage pipeline: collect counts, then factorize.

Can we learn the same dense representations directly, in a single pass over text?

Learning Vectors Directly: Word2Vec

Instead of counting contexts and then compressing, train a model to *predict* contexts directly. The vectors that emerge from this prediction task will encode the same co-occurrence structure — but learned online, in a single pass, without ever building the full matrix.

Each word w is assigned a trainable vector $\mathbf{v}_w \in \mathbb{R}^d$. All vectors are collected into an

embedding matrix:

$$E \in \mathbb{R}^{d \times V},$$

where column i holds the vector for the i-th vocabulary word. Given a word w with one-hot vector $\mathbf{e}_w$, its embedding is:

$$\mathbf{v}_w = E\,\mathbf{e}_w = E_{:,\,i(w)}.$$

This is just column selection — the one-hot vector picks out a single column of E. The matrix E is learned end-to-end during training.

A note on terminology. Each word appears in two roles: sometimes it is the word being predicted *from* (the center word), sometimes it is the word being predicted (the context word). We give each word two separate vectors — $\mathbf{v}_w$ for when it acts as center, $\mathbf{u}_w$ for when it acts as context — and learn both independently. This works better in practice than sharing a single vector.

The Skipgram Objective

The training task is:

given a center word, predict which words appear nearby.

For a sentence like "I drank hot coffee this morning," treating **coffee** as the center word with window size $k = 2$ generates the training pairs:

$$(\text{coffee, hot}), \quad (\text{coffee, drank}), \quad (\text{coffee, this}), \quad (\text{coffee, morning}).$$

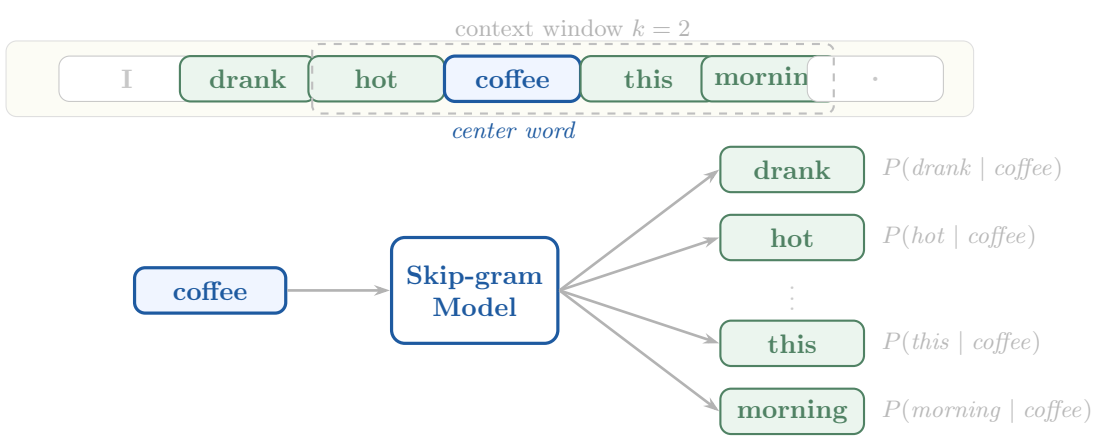

Figure 1.7: The Skip-gram model: given center word *coffee*, the model predicts each context word within a window of size $k = 2$. The training pairs are (*coffee*, *drank*), (*coffee*, *hot*), (*coffee*, *this*), and (*coffee*, *morning*).

For each pair, the model should assign high probability to the observed context word and low probability to words that did not appear. To score how compatible a center word c and context word o are, we use their dot product:

$$\mathbf{u}_o^\top \mathbf{v}_c.$$

When two vectors point in the same direction, their dot product is large. Training will push the vectors of co-occurring words to align — which is exactly what encodes "these words belong together." Converting scores to probabilities via softmax:

$$p(o \mid c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \mathcal{V}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}.$$

Training maximizes this probability across all observed center–context pairs in the corpus.

What the Gradient Actually Does

The gradient of the log-probability with respect to the center vector $\mathbf{v}_c$ is:

$$\nabla_{\mathbf{v}_c} \log p(o \mid c) = \underbrace{\mathbf{u}_o}_{\substack{\text{observed}\\ \text{context}}} - \underbrace{\sum_{x \in \mathcal{V}} p(x \mid c)\, \mathbf{u}_x}_{\substack{\text{model's current}\\ \text{expected context}}}.$$

> This is a correction signal. The update moves $\mathbf{v}_c$ **toward the vector of the word that actually appeared** and **away from the weighted average of all words the model currently predicts**. If the model is already confident about the right context word, the correction is small. If it is confidently wrong, the correction is large.

Repeat this over every center–context pair in the corpus — billions of small corrections, each one nudging vectors toward or away from each other.

> Words that share contexts get pulled toward each other. Words that never share contexts drift apart. The geometry of the embedding space becomes a map of linguistic relatedness.

CBOW:

Skipgram asks: *given this word, what surrounds it?* **Continuous Bag of Words (CBOW)** flips the question: *given the surroundings, what is the missing word?* The context vectors are averaged and fed through the same softmax to predict the center.

Averaging multiple context signals makes CBOW faster and more stable for common words. But averaging also smooths out the contribution of any individual context word — rare words, which appear in few contexts, receive weaker gradient signal. Skipgram, treating each center–context pair as a separate training example, gives rare words more updates.

	Skipgram	CBOW
Training speed	Slower	Faster
Rare words	Better	Worse
Frequent words	Worse	Better
Analogy tasks	Better	Worse

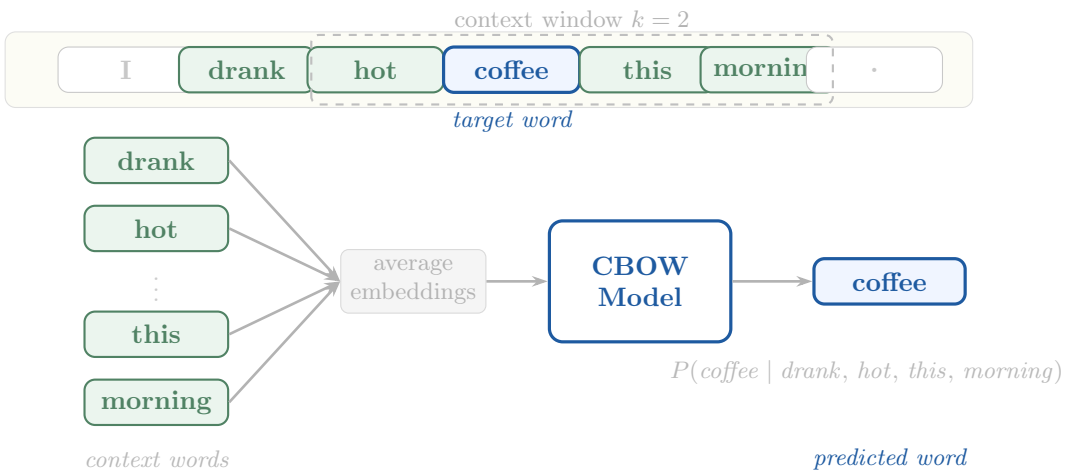

Figure 1.8: The CBOW model: all context words within a window of size $k = 2$ — *drank*, *hot*, *this*, *morning* — are averaged and used to predict the target word *coffee*.

What the Geometry Encodes

Once training converges, something unexpected emerges: the vector space has *linear structure*. Semantic relationships are encoded as consistent geometric offsets.

The canonical example:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}.$$

> No one encoded this relationship explicitly. It emerges from the training objective alone.

The vector $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}}$ encodes "royalty, with gender factored out." Adding $\mathbf{v}_{\text{woman}}$ applies that concept to the female case.

The same pattern holds across many relationship types:

Emergent Linear Structure

Relationship	Vector arithmetic
Verb tense	$\mathbf{v}_{\text{walking}} - \mathbf{v}_{\text{walk}} \approx \mathbf{v}_{\text{swimming}} - \mathbf{v}_{\text{swim}}$
Country–capital	$\mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} \approx \mathbf{v}_{\text{Berlin}} - \mathbf{v}_{\text{Germany}}$
Comparative	$\mathbf{v}_{\text{bigger}} - \mathbf{v}_{\text{big}} \approx \mathbf{v}_{\text{taller}} - \mathbf{v}_{\text{tall}}$

This linearity is what makes word vectors useful beyond lookup tables. A model that has learned "Paris is the capital of France" implicitly learns the *concept* of a capital city as a vector offset. It can then answer "what is the capital of Germany?" by arithmetic, even if it never saw that question during training. The geometry generalizes.

What We Are Really Trying to Do

A word is a symbol. Behind it sits a concept — a thought, a feeling, a texture of meaning — that exists in the mind before language does. When we speak or write, we reach into a shared vocabulary and try to compress that concept into symbols, hoping it reassembles correctly in someone else's mind.

There is a useful analogy here, not a precise biological claim, but an intuition worth holding. When you encounter a concept — "coffee," "loss," "home" — something happens in the brain that is not a single neuron firing but a *combination*: a pattern spread across many neurons, each partially active, whose collective state encodes the meaning. Change the context and the pattern shifts. "Bank" beside "river" and "bank" beside "investment" are the same symbol but, in some sense, a different mental event. Meaning lives not in the symbol but in the pattern it evokes.

A word vector borrows this structure. Instead of neurons, dimensions. Instead of activation levels, real-valued coordinates. A dense vector $\mathbf{v} \in \mathbb{R}^d$ is a pattern across d numbers — some large, some small, some negative — whose combination encodes a concept in a way that no single number could.

Word2Vec was a first attempt to learn these patterns from data. It worked surprisingly well, but with one fundamental limitation: each word received a single fixed vector regardless of context. "Bank" always activated the same combination of dimensions, whether the sentence was about rivers or money.

What we actually want is a representation that shifts with context — the same symbol producing a different pattern depending on what surrounds it. Building that is the problem we turn to next.

Relaxing orthogonality increases capacity. To see the effect concretely, consider an embedding space of dimension $n = 1000$. If we require vectors to be exactly orthogonal, then at most 1000 independent directions can exist. Now relax this constraint slightly and allow vectors to be nearly orthogonal, meaning their pairwise inner products

satisfy

$$|\langle v_i, v_j \rangle| \leq \varepsilon.$$

High-dimensional geometry shows that the number of such vectors scales roughly as

$$M \approx 2^{\Theta(\varepsilon^2 n)}.$$

For example, if we allow a moderate deviation from orthogonality corresponding to angles of about 85° (so $\varepsilon \approx 0.087$), then $\varepsilon^2 n \approx 7.5$. This implies that the number of nearly independent directions can be on the order of

$$M \sim 2^{7.5} \approx 180$$

times larger than the strictly orthogonal limit. Instead of only 1000 independent directions, the space may support on the order of hundreds of thousands of approximately independent ones. As the dimension increases further, this growth becomes extremely rapid.

The implication for word vectors is important: embedding spaces do not need one dimension per concept. By tolerating small overlap between directions, a moderate-dimensional space can represent vastly more words or semantic features than its raw dimensionality would suggest, while still keeping them geometrically distinguishable.

In practice, we no longer construct word vectors using skip-gram, CBOW, GloVe, or any explicit co-occurrence objective. We initialize E randomly and let the model learn it end-to-end — gradient descent discovers the same structure implicitly, as a byproduct of learning to do something harder. What these methods gave us is not a recipe but an understanding: dense vectors can encode meaning, and context is how meaning is learned.

Bridging NADE to RNN. NADE already contains the core idea that later leads to recurrent models. Instead of recomputing a new network for every conditional distribution, it incrementally updates a running quantity,

$$a_t = a_{t-1} + W_{:,t-1} x_{t-1},$$

which accumulates information about previously observed variables. Each conditional distribution is produced from this accumulated pre-activation.

From one perspective, NADE behaves as if each variable had its own neural network for conditional modeling, but parameter sharing allows these networks to be implemented efficiently through incremental updates. The accumulated vector therefore acts as a primitive form of memory of previous variables – words.

However, the accumulation rule in NADE is limited in two ways. First, the update is purely additive and linear: the model learns *what* to add at each step via the weight columns $W_{:,t-1}$, but not *how* the accumulated state itself should evolve in response to its current value. Second,

images/chapter2_images/nade_rnn.png

Figure 1.9: NADE architecture with pre-activation accumulation

the weight columns are position-specific, so the update rule is not shared across steps; the model learns a separate transformation for each position in the sequence.

Recurrent neural networks address both limitations through one conceptual change: the state update becomes a learned function shared across all positions. Instead of a fixed additive update with position-specific weights, we introduce a learned state transition,

$$h_t = f(h_{t-1}, x_t),$$

where the same function f is applied at every step. The hidden state h_t is a fixed-dimensional representation summarizing everything observed so far.

In this view, RNNs can be understood as replacing NADE's handcrafted, position-dependent accumulation rule with a learnable, nonlinear, position-independent update that defines how information is carried forward.

1.6.2 RNNs: The Hidden State as Context

Recurrent Neural Networks (RNNs) were introduced to overcome a core limitation of feed-forward models: the inability to naturally represent sequential structure.

Many forms of data are inherently ordered. Language unfolds token by token, audio evolves continuously over time, and even images may be interpreted as ordered sequences of pixels or patches. In such settings, the prediction at time t should depend not only on the current input x_t but also on the history $x_{<t}$.

A standard neural network treats inputs independently and therefore has no mechanism for remembering past observations. RNNs address this by introducing a *hidden state*, a vector that is updated at each time step and acts as a compressed representation of everything seen so far. In this sense, the hidden state provides the contextual summary needed for sequential prediction.

Definition 1.8: Recurrent Neural Network (RNN)

A **Recurrent Neural Network** processes a sequence one element at a time, updating a hidden state h_t according to

$$h_t = \phi_1(W_{hh}h_{t-1} + W_{xh}x_t + b_h),$$

and producing an output

$$\hat{y}_t = \phi_2(W_{hy}h_t + b_y),$$

which is typically interpreted as the parameterization of a conditional distribution

$$p(y_t \mid h_t).$$

The parameters W_{hh}, W_{xh}, W_{hy} and biases b_h, b_y are shared across all time steps, allowing sequences of arbitrary length to be processed with a constant number of parameters.

images/chapter2_images/rnn.png

Figure 1.10: Recurrent neural network. The same transformation is reused at every time step.

The key idea is simple: rather than explicitly storing all previous inputs, the model learns a function that compresses past information into a fixed-dimensional state. This compact representation enables flexible sequence modeling — but it also introduces a fundamental optimization challenge.

Gradient Flow Through Time

Training an RNN requires learning how information should persist across many steps. This immediately raises a question: *how does learning signal propagate backward through a long sequence?*

Because each hidden state depends on the previous one, gradients must travel through the entire temporal chain. The behavior of this backward flow determines whether the model can learn long-range dependencies.

Assume a sequence of length T, with a loss defined at each step:

$$L_{\text{sequence}} = \sum_{t=1}^{T} L_t.$$

To study gradient propagation, it is sufficient to examine the contribution of a single future loss term L_k $(k \geq t)$ to an earlier hidden state h_t. The full gradient is obtained by summing such contributions.

Our goal is therefore to understand how

$$\frac{\partial L_k}{\partial h_t}$$

changes as the temporal distance $(k - t)$ grows.

Backpropagation Through Time Since hidden states form a temporal chain

$$h_t \to h_{t+1} \to \cdots \to h_k,$$

the chain rule yields

$$\frac{\partial L_k}{\partial h_t} = \frac{\partial L_k}{\partial h_k} \prod_{j=t+1}^{k} \frac{\partial h_j}{\partial h_{j-1}}.$$

Thus, gradients are obtained by repeatedly multiplying Jacobians — one for each recurrent step. The central question becomes: what happens when this multiplication is repeated many times?

Jacobian of a Recurrent Step From the update equation,

$$h_j = \phi_1(W_{hh} h_{j-1} + W_{xh} x_j + b_h),$$

the Jacobian with respect to the previous hidden state is

$$\frac{\partial h_j}{\partial h_{j-1}} = D_j W_{hh},$$

where

$$D_j = \operatorname{diag}\big(\phi_1'(a_j)\big), \qquad a_j = W_{hh} h_{j-1} + W_{xh} x_j + b_h.$$

Substituting,

$$\frac{\partial L_k}{\partial h_t} = \frac{\partial L_k}{\partial h_k} \prod_{j=t+1}^{k} D_j W_{hh}.$$

The same transformation is therefore applied repeatedly as gradients move backward through time. This repeated reuse of the recurrent dynamics is the source of both the power and the instability of RNN training.

A Linearized View: Why Gradients Grow or Shrink To build intuition, we first isolate the effect of recurrence itself. Suppose the activation derivatives vary slowly across time, so that locally the network behaves approximately linearly. In this regime, we may treat

$$D_j \approx I,$$

not as an exact assumption, but as a simplifying lens that reveals the dominant dynamics of recurrence.

Under this approximation,

$$\frac{\partial L_k}{\partial h_t} \approx \frac{\partial L_k}{\partial h_k} W_{hh}^{(k-t)}.$$

The problem reduces to repeated powers of the recurrent matrix. If W_{hh} is diagonalizable,

$$W_{hh} = Q\Lambda Q^{-1}, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n),$$

then each component aligned with eigenvector i scales as

$$\lambda_i^{(k-t)}.$$

This immediately reveals the core phenomenon:

- If $|\lambda_i| < 1$, gradients decay exponentially.
- If $|\lambda_i| > 1$, gradients grow exponentially.

Stable propagation requires eigenvalues whose magnitudes remain close to 1, a condition that is difficult to maintain during training.

Returning to the Nonlinear Case The linearized picture provides intuition, but the full network also includes activation derivatives. Taking norms and using submultiplicativity,

$$\left\| \frac{\partial L_k}{\partial h_t} \right\| \leq \left\| \frac{\partial L_k}{\partial h_k} \right\| \prod_{j=t+1}^{k} \|D_j\| \, \|W_{hh}\|.$$

For common activations such as sigmoid or tanh,

$$|\phi_1'(x)| \leq \alpha < 1,$$

which implies $\|D_j\| \leq \alpha$. Activation derivatives therefore introduce an additional contractive effect, reinforcing the tendency for gradients to shrink over long horizons.

Combined with the recurrent dynamics, this leads to two characteristic pathologies:

- **Vanishing gradients:** early time steps receive almost no learning signal, making long-term dependencies difficult to learn.
- **Exploding gradients:** gradient norms grow uncontrollably, producing unstable parameter updates.

Why Exploding Gradients Are Easier to Handle In practice, exploding gradients are more manageable than vanishing ones. Explosions manifest as sudden large norm spikes, making them detectable during training. A simple and widely used remedy is *gradient clipping*, which rescales gradients whenever they exceed a threshold τ:

$$g \leftarrow g \cdot \frac{\tau}{\|g\|} \qquad \text{if } \|g\| > \tau.$$

Vanishing gradients, by contrast, are silent: the learning signal simply disappears, leaving the model unable to adjust parameters that influence distant past states.

> Backpropagation through time repeatedly applies the same recurrent transformation. Small deviations from unit magnitude accumulate exponentially: some directions shrink, others grow. As sequence length increases, gradients either vanish or explode, making long-range learning unstable in vanilla RNNs.

The limitation of vanilla RNNs is not representational power but optimization.

A recurrent network can, in principle, model complex sequential dependencies using a fixed set of shared parameters. In practice, however, learning is constrained by unstable gradient flow and by the need to compress an ever-growing history into a finite-dimensional hidden state. These two issues — fragile optimization and memory compression — motivate the architectural developments that follow.

LSTM and GRU: Fixing the Memory

The core issue is that memory is updated multiplicatively at every step, so gradients must pass through repeated products. Long-range dependencies — the kind that matter most in language — are exactly what vanilla RNNs cannot learn. The LSTM was designed specifically to fix this.

Definition 1.9: Long Short-Term Memory (LSTM)

The **LSTM** introduces a **cell state** c_t alongside h_t, controlled by three learned gates:

$$\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{(forget gate: what to erase from } c_t\text{)} \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{(input gate: what new info to write)} \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(output gate: what to expose as } h_t\text{)} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c) \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}$$

The cell state c_t is updated by **addition** rather than multiplication. Gradients flowing through c_t do not shrink exponentially, allowing the LSTM to learn dependencies spanning hundreds of steps. Both h_t and c_t are deterministic.

The **Gated Recurrent Unit (GRU)** is a streamlined variant that merges c_t and h_t into a single state with two gates instead of three. Computationally cheaper, and in practice often

matches LSTM performance.

LSTMs and GRUs dominated sequence modeling from their introduction through the mid-2010s, producing strong results in machine translation, speech recognition, and language modeling. Their fundamental limitation remains: processing position t requires the output of position $t-1$. Training is inherently sequential and cannot be parallelized across the time dimension.

Linear accumulation revisited. The LSTM brings back an idea that first appeared in NADE: a linear additive path for carrying information forward. RNNs replaced linear accumulation with nonlinear state updates to gain expressive power, but this came at a cost: gradients shrink as they pass through many transformations, making long-range dependencies hard to learn. The LSTM addresses this by separating two roles that vanilla RNNs conflate. The hidden state handles nonlinear computation at each step, while the cell state follows a controlled additive update, gated by what the model has learned to remember or forget, giving gradients a stable path to flow backward across many steps. The key difference from NADE is that the linearity is not fixed: the forget and input gates decide at each step how much of the additive path to use. Uncontrolled linearity limits what a model can represent; controlled linearity, combined with nonlinear computation elsewhere, is what makes long-range memory possible.

Bidirectional RNNs

A standard RNN reads left to right. At position t, the hidden state h_t summarizes everything seen so far — $x_1, \ldots, x_t$ — and nothing after it. For generation this is unavoidable: you cannot condition on tokens you have not yet produced. But when the full input sequence is already available, discarding the right context is a waste.

Consider labeling each word in a sentence with its part of speech. The word "bank" is ambiguous in isolation, but not in "the bank of the river" versus "the bank approved the loan." A left-to-right RNN processing "the bank" has not yet seen the disambiguating context. A model that could see both directions simultaneously would have no such problem.

A **bidirectional RNN** runs two RNNs over the same sequence:

$$\overrightarrow{h}_t = \mathrm{RNN}\Big(x_t,\ \overrightarrow{h}_{t-1}\Big) \quad \text{(left to right)} \tag{1.1}$$

$$\overleftarrow{h}_t = \mathrm{RNN}\Big(x_t,\ \overleftarrow{h}_{t+1}\Big) \quad \text{(right to left)} \tag{1.2}$$

The two hidden states are concatenated at each position:

$$h_t = \Big[\ \overrightarrow{h}_t\ ;\ \overleftarrow{h}_t\ \Big] \tag{1.3}$$

Now h_t encodes the full sequence context around position t — everything to its left and everything to its right. This makes bidirectional RNNs the natural choice whenever the task is to *encode* an input rather than *generate* an output. We will see this distinction become critical in the encoder-decoder architecture below.

Stacked RNNs and LSTMs

A single RNN layer maps an input sequence to a sequence of hidden states. Those hidden states are themselves a sequence — so there is nothing stopping us from feeding them into

another RNN.

$$h_t^{(1)} = \mathrm{RNN}^{(1)}\left(x_t,\, h_{t-1}^{(1)}\right) \tag{1.4}$$

$$h_t^{(2)} = \mathrm{RNN}^{(2)}\left(h_t^{(1)},\, h_{t-1}^{(2)}\right) \tag{1.5}$$

$$\vdots$$

$$h_t^{(L)} = \mathrm{RNN}^{(L)}\left(h_t^{(L-1)},\, h_{t-1}^{(L)}\right) \tag{1.6}$$

Lower layers capture local, surface-level patterns; upper layers build more abstract representations over longer spans. The final output at each timestep is $h_t^{(L)}$, the top-layer hidden state. The same principle applies directly to LSTMs — each layer receives the hidden states of the layer below as its input sequence, and maintains its own cell state independently.

Sequence-to-Sequence Learning: The Encoder-Decoder Architecture

Every model so far models a single sequence: $p(x_1, \ldots, x_n)$. But many tasks involve two sequences — an input and an output that are related but distinct. Machine translation is the clearest example: given an English sentence of length n, produce a German sentence of length m, where $n \neq m$ in general and neither is known in advance.

What we need is a conditional model:

$$p(y_1, \ldots, y_m \mid x_1, \ldots, x_n) \tag{1.7}$$

The natural way to build this is to split the job in two. One network reads the input and builds a representation of it. Another network generates the output, conditioned on that representation. This is the **encoder-decoder** architecture.

Encoder. The encoder reads the source sequence $x_{1:n}$ and transforms it into a single fixed-length representation called the **context vector**. At each step the encoder updates its hidden state using the current input token and the previous state:

$$h_t = f_W(x_t, h_{t-1}) \tag{1.8}$$

After the final token is processed, the last hidden state becomes the summary of the entire input:

$$c = h_n \tag{1.9}$$

Decoder. The decoder generates the output sequence one token at a time. Its hidden state depends on the previously generated token, the previous decoder state, and the context vector produced by the encoder:

$$s_t = g_U(y_{t-1}, s_{t-1}, c) \tag{1.10}$$

The probability of the next token is obtained by applying a softmax to the decoder state:

$$p(y_t \mid y_{<t}, x_{1:n}) = \mathrm{softmax}(W s_t + b) \tag{1.11}$$

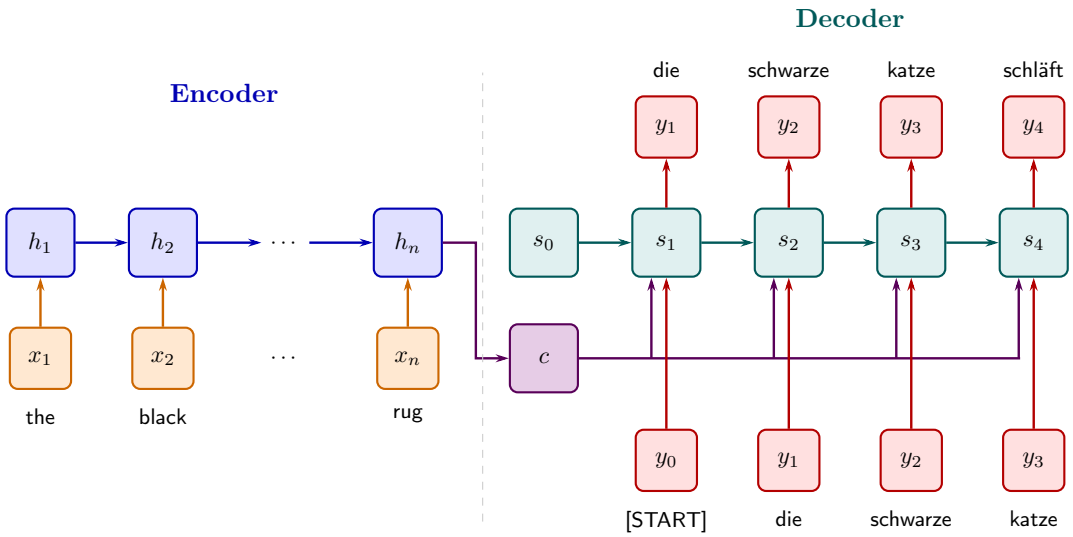

Figure 1.11: Sequence-to-sequence encoder–decoder architecture translating the English sentence "the black cat sleeps on the rug" into German.

The model therefore factorizes the conditional probability of the output sequence as

$$p(y_{1:m} \mid x_{1:n}) = \prod_{t=1}^{m} p(y_t \mid y_{<t}, x_{1:n}) \tag{1.12}$$

The bottleneck. There is a quiet problem in this architecture. The context vector c is fixed-size. It does not matter whether the source sentence has 5 words or 50 — the encoder must compress everything into the same vector. For short sentences, this works well. For long sentences, it does not: information gets squeezed out, and translation quality degrades visibly with source length.

The problem is not the LSTM — a more powerful encoder will not fix it. The problem is structural. A single vector is a hard bottleneck, and no encoder, however deep, can overcome a fixed-size information channel when the input is arbitrarily long.

The fix is to stop forcing the decoder to read from a single summary. Instead, let the decoder look back at *all* the encoder hidden states $h_1, \ldots, h_n$ and decide, at each generation step, which parts of the source to focus on. That mechanism is **attention**, and it is the subject of the next section.

1.6.3 Attention

Dynamic Retrieval: Let the Decoder Look Back

The root cause of the seq2seq bottleneck is that the decoder is cut off from the source sentence. Once the encoder finishes, the entire input must be reconstructed from a single fixed vector.

The natural fix is simple: keep all the encoder hidden states available and let the decoder look at them directly.

Let the encoder produce a sequence of hidden states

$$(h_1, h_2, \ldots, h_{T_x}),$$

where each h_j represents the j-th English word in the context of its neighbours. These vectors are not discarded after encoding. Instead, the decoder can access all of them at every step.

When generating the i-th German word, the decoder constructs a *dynamic context vector* by taking a weighted combination:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij}\, h_j.$$

The weights α_{ij} determine how relevant each English word is to the current decoding step. They are computed by scoring the compatibility between the decoder state s_{i-1} and each encoder state h_j, then normalising via softmax:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}.$$

The weights form a probability distribution over the source positions. When the decoder generates the German verb *schläft*, it places high weight on the English word *sleeps* and low weight on unrelated words such as *the* or *rug*. When generating a noun, the focus shifts accordingly.

The retrieval is dynamic and input-dependent. At each decoding step the model decides which parts of the source sentence matter most.

This mechanism is *attention*.

Definition 1.10: Attention

Attention converts a set of static representations into a dynamic contextual representation by computing a relevance-weighted mixture of those representations.

The bottleneck disappears. The decoder no longer extracts everything from a single vector; it selectively retrieves information from the entire encoded sequence. This raises a key question:

How should compatibility be scored? Everything hinges on computing the compatibility score e_{ij} well. This is the central design question of attention.

First Design: Score with a Network

The most expressive option is to learn a small neural network that takes the decoder state s and encoder state h and outputs a score:

$$e = v^\top \tanh(W_1 s + W_2 h).$$

The network learns directly from translation data what compatibility means. It can capture complex nonlinear interactions between the decoder and encoder representations.

This approach works well, but it has a structural cost. The network must be evaluated for *every pair* of decoder and encoder positions. Because decoder states are generated sequentially, the computation cannot be parallelised easily. For long sentences this becomes expensive.

Second Design: Score with a Dot Product

A much simpler alternative is to replace the neural network with a dot product:

$$e = s^\top h.$$

The dot product requires no additional parameters and is extremely cheap to compute. Surprisingly, it performs competitively in practice.

But taking dot products between raw hidden states introduces two problems.

The relevance problem. A dot product measures raw similarity between vectors. Two states will score highly if their representations happen to be close, even if the words are not actually relevant to each other in this particular sentence. Similarity is not necessarily relevance.

The scale problem. When the hidden state dimension d is large, dot products tend to grow in magnitude proportionally to d. This pushes the softmax into saturation, where gradients become very small and training slows down.

Both issues arise because the vectors are being compared in the wrong space.

The Fix: Project Before You Compare

The solution is to project the states into a learned interaction space before computing the dot product.

Consider the English sentence

"The black cat sleeps on the rug."

Suppose the decoder has produced the German phrase *"Die schwarze Katze"* and must now generate the next word. It needs to locate the English verb *sleeps*.

The decoder state s encodes everything known so far about the German sentence. The encoder states $h_1, \ldots, h_n$ encode the English words. But these vectors live in different representation spaces. Comparing them directly is like comparing measurements expressed in different units.

The fix is to learn projections that place them in a common space.

The decoder state is projected into a **query**:

$$q = sW_Q.$$

Each encoder state is projected into a **key**:

$$k_j = h_j W_K.$$

Both projections map into the same d_k-dimensional space. The compatibility score becomes

$$e_j = \frac{q^\top k_j}{\sqrt{d_k}}.$$

The scaling factor $\sqrt{d_k}$ prevents dot products from growing too large and stabilises training.

Now the decoder query for the next German verb aligns strongly with the key of the English word *sleeps* and weakly with unrelated words.

The projections solve **Relevance**: the model learns how queries and keys should align.

What Gets Retrieved: The Value Projection

Once attention weights are computed, the model takes a weighted sum of vectors from the encoder. But a weighted sum of *what*?

Using the key vectors themselves would mix two roles that should remain separate: how a word is *found* and what it *contributes* once found.

Keys are optimised for discoverability. But the decoder ultimately needs the meaning of the word.

To separate these roles, each encoder state is also projected into a **value** vector:

$$v_j = h_j W_V.$$

Queries and keys determine how much each word contributes. Values determine what information is actually retrieved.

The resulting context vector is

$$c_i = \sum_j \alpha_{ij} v_j.$$

The word *sleeps* is located because its key matches the query. What is retrieved, however, is its value — a representation encoding its semantic and grammatical content.

What projection means geometrically For each English word the encoder produces a hidden state h_j. This state is projected into a key k_j, while the decoder state s is projected into a query q.

Seven English words therefore produce seven keys. The decoder produces one query. Their dot products measure how well each source word matches the decoder's current need.

Softmax converts these scores into weights, and the decoder retrieves a weighted mixture of value vectors.

Geometrically, the projections isolate the components of the representations that are relevant for interaction. The query isolates the features that express what the decoder is searching for. The keys isolate the features that advertise what each word contains.

The hidden states themselves are simply the vectors that feed this interaction.

This observation leads to an important question. Attention does not depend on how the vectors $h_1, \ldots, h_{T_x}$ were produced. It only requires that each position in the sentence is associated with a vector that can be projected into queries, keys, and values.

In the encoder–decoder architecture these vectors come from a recurrent encoder:

$$h_j = f(x_j, h_{j-1}).$$

But once attention allows the decoder to retrieve information directly from any position in the source sentence, the role of the recurrent chain changes. In the original encoder, the

dependence on the previous state was necessary because contextual information had to be carried forward through the sequence. Each hidden state inherited information from the previous one so that earlier words could influence later representations.

Attention removes this requirement. The decoder can now access every position in the source sentence directly and retrieve whatever information it needs through the attention weights. Context no longer has to be transported step by step through the recurrent chain.

Once this mechanism exists, the dependence on the previous hidden state becomes unnecessary. The representation associated with each word no longer needs to carry the entire sentence history forward. The encoder transformation can therefore be simplified to

$$h_j = f(x_j).$$

At this point the role of f itself becomes unclear. Word embeddings x_j are learned parameters of the model. If a particular vector representation of a word is useful for the attention mechanism, the training process can move the embedding in that direction.

In effect, the embedding itself can serve as the vector that attention operates on. The model can simply learn the embedding of each word to be the representation that best supports attention. We can therefore identify the hidden state with the word vector itself,

$$h_j = x_j.$$

The same reasoning applies on the decoder side. The decoder state s was previously the representation used to form queries. But if the model already maintains a vector representation for each generated token, that representation can directly serve as the query source. Attention therefore operates on the word representations themselves rather than on hidden states produced by a recurrent network.

Once we adopt this view, attention can be written directly in terms of the word representations in the encoder and decoder sequences.

The decoder representation at position i produces a query, while the encoder representations produce keys and values:

$$q_i = y_i W_Q, \qquad k_j = x_j W_K, \qquad v_j = x_j W_V.$$

The decoder query is compared with all encoder keys to determine which source positions are relevant, and the resulting weights are used to retrieve a weighted combination of the corresponding values. Written in matrix form, this mechanism becomes the standard definition of scaled dot-product attention.

Definition 1.11: Scaled Dot-Product Attention

For queries Q, keys K, and values V stacked into matrices:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

- **Query** (Q): what each position is looking for.

- **Key** (K): what each position is advertising it contains.
- **Value** (V): what each position contributes once found.
- $\sqrt{d_k}$: scaling factor that prevents dot products from growing large and saturating the softmax.

The output at each position is a weighted sum of values, where the weights are determined entirely by query–key compatibility.

The mechanism developed so far is *cross-attention*: the German decoder attends to the English encoder. The query comes from the German side; the keys and values come from the English side. The two sequences are different.

Definition 1.12: Cross-Attention

Cross-attention applies the attention operator to two different sequences. Queries are produced from the decoder representations, while keys and values are produced from the encoder representations.
Let Y be the matrix of decoder representations and X the matrix of encoder representations. Cross-attention is defined as

$$\text{CrossAttention}(Y, X) = \text{Attention}(YW_Q,\ XW_K,\ XW_V).$$

Each decoder position forms a query that searches over the encoder representations to retrieve the information most relevant to the next prediction.

Self-Attention: Turning the Mechanism Inward

But now consider the English sentence itself, before translation begins. The meaning of each English word depends on the words around it. The word *bank* means something different beside *river* than beside *loan*. A recurrent encoder captures this by passing information forward step by step, but this forces early words to carry their influence through a long chain of hidden states to reach distant ones — a slow, sequential, and lossy process.

The same attention mechanism can be applied within a single sequence. Each English word produces its own query, key, and value, and attends over all other words in the same sentence. This is *self-attention*: the sequence attending to itself.

The effect is significant. Any two positions interact directly in a single operation — *bank* can immediately attend to *river* or *loan* to resolve its meaning, without the relationship having to propagate through every intermediate hidden state. Long-range dependencies are no harder to capture than short-range ones, and the entire computation runs in parallel.

Definition 1.13: Self-Attention

The queries, keys, and values are all derived from the same input sequence:

$$\text{SelfAttention}(X) = \text{Attention}(XW_Q,\ XW_K,\ XW_V).$$

Each position attends over all other positions in the same sequence, allowing direct interaction between any two tokens regardless of their distance. This enables the model to refine the representation of each token using information from the entire sentence.

Self-attention in the encoder allows every position to attend to every other position in the sentence. The decoder, however, generates the target sentence one token at a time. When predicting the token at position i, the model must not see future tokens.

Masked Self-Attention

In the encoder, every token can attend to every other token in the sentence because the entire input is known in advance.

The decoder is different. It generates the German sentence one word at a time. Suppose the model has produced

Die schwarze Katze

and is about to generate the next word *schläft*. When computing the representation for the next position, the decoder may attend to the words that already exist — *Die*, *schwarze*, and *Katze* — but it must not look ahead to words that have not yet been generated, such as *auf*, *dem*, or *Teppich*. Allowing this during training would reveal the future and make the task trivial.

To enforce this causal ordering, attention to future positions is masked.

Definition 1.14: Masked Self-Attention

Masked self-attention computes attention within a sequence while preventing positions from attending to future tokens.

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V$$

The mask matrix M is

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

so that token i can attend only to tokens at positions $j \leq i$.

Multiple Heads: Multiple Notions of Relevance

A single set of projections W_Q, W_K, W_V learns one notion of relevance. At each decoding step, the attention weights form a single distribution over the source sequence — one joint decision about what to retrieve. The problem is that relevance is not one thing.

Even within a single sentence, generating one word can require answering several structurally different questions simultaneously. When the German decoder generates *schläft*, it needs

to know: *what is the verb?* (to find *sleeps*), *what is the subject?* (to enforce grammatical agreement with *cat*), and *what is the object?* (to understand the full predicate). These are not the same question. A single attention distribution must place its weight somewhere — it cannot simultaneously peak at *sleeps*, *cat*, and *rug* without diluting each. The model is forced to average over relationships that should be kept separate.

The solution is to run H independent attention heads in parallel, each with its own projections $W_Q^{(h)}$, $W_K^{(h)}$, $W_V^{(h)}$:

$$\text{head}_h = \text{Attention}(XW_Q^{(h)}, XW_K^{(h)}, XW_V^{(h)})$$

Each head operates in its own subspace and learns its own notion of what makes two positions relevant to each other. One head may learn to track syntactic agreement, another lexical correspondence, another long-range coreference. Because they run in parallel on the same input, the model simultaneously captures all the relationships a single head would be forced to trade off. Every head runs on every input, every time — there is no routing and no competition. The specialisation emerges entirely from training.

The outputs of all heads are concatenated and passed through a final projection W^O:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W^O.$$

The final projection does not merely stack the outputs — it learns to mix information across heads, so that what one head noticed about syntax and another noticed about semantics can be combined into a single representation that reflects both.

Definition 1.15: Multi-Head Attention (MHA)

Let $X \in \mathbb{R}^{n \times d_{\text{model}}}$ be a matrix of input representations. Multi-head attention runs H attention heads in parallel, each with its own learned projections:

$$\text{head}_h = \text{Attention}(XW_Q^{(h)},\ XW_K^{(h)},\ XW_V^{(h)}),$$

where

$$W_Q^{(h)}, W_K^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_k}, \qquad W_V^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

The outputs of all heads are concatenated and projected back to the model dimension:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\, W^O,$$

with

$$W^O \in \mathbb{R}^{Hd_v \times d_{\text{model}}}.$$

- Each head operates in a reduced-dimensional subspace with $d_k = d_v = d_{\text{model}}/H$.
- Concatenating the H heads restores the original dimensionality $Hd_v = d_{\text{model}}$.
- Because each head works in a smaller space, the total computational cost remains comparable to that of single-head attention with full dimension d_{model}.
- Each head learns an independent notion of relevance.
- W^O mixes information across heads into a single unified representation.

Recurrence was never the goal. Looking back at the full arc: recurrence was not a fundamental principle. It was a mechanism for propagating dependencies step by step because no direct alternative existed. Attention is that alternative — any two positions can interact directly, without a sequential chain between them.

Cross-attention lets the German decoder retrieve information from the English encoder. Self-attention lets a sequence refine its own representation by allowing its tokens to interact with one another. In the decoder this interaction must respect causal order: when predicting the word at position t, the model must rely only on the words that have already been generated. This is enforced through *masked self-attention*, which blocks attention to future positions.

Both cross-attention and self-attention rely on the same underlying mechanism: queries, keys, and values are computed, compatibility scores determine which positions are relevant, and the resulting information is aggregated.

The pieces are now in place. The Transformer is the architecture that builds entirely around this idea, replacing recurrence with attention throughout and surrounding it with the components needed to make attention work at scale.

1.6.4 Transformer

What the Recurrence Was Providing

Token embeddings carry over unchanged to the Transformer. Given a tokenized sequence $(x_1, x_2, \ldots, x_N)$, the embedding matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$$

maps tokens to dense vectors, producing

$$X \in \mathbb{R}^{N \times d_{\text{model}}}.$$

What recurrence uniquely provided was an implicit encoding of token order. Recurrent models process tokens sequentially, so the representation at position t is built from the representation at $t-1$. Through this chain the model learns the order of the sequence.

Self-attention does not have this property. It processes all tokens simultaneously and is therefore permutation-invariant: it has no intrinsic notion of which token came first.

Removing recurrence therefore removes the implicit encoding of order. The Transformer restores this information by adding a **positional encoding** to each token embedding:

$$X^{(0)} = \mathit{TokenEmbedding} + \mathit{PositionEmbedding}.$$

Each representation now contains both the identity of the token and its position in the sequence.

Re-engineering the Decoder

In the recurrent architecture, the decoder updated its internal state according to

$$s_t = f(s_{t-1},\, y_{t-1},\, c_t).$$

This single equation hides three distinct roles.

- **Target history.** The hidden state s_{t-1} carried a summary of previously generated tokens.
- **Previous token.** The embedding of y_{t-1} provided the most recent symbol produced by the model.
- **Source context.** The vector c_t was dynamically retrieved from the encoder using attention.

The Transformer keeps these roles but implements them without recurrence.

Target history. Instead of storing history in a single evolving state, the Transformer allows each token to directly attend to earlier tokens in the sequence. Masked self-attention computes a relevance-weighted mixture of representations from the prefix, allowing every token to incorporate the information it needs from previously generated tokens.

Source retrieval. Information from the source sentence is incorporated through cross-attention. Decoder representations produce queries, while encoder representations provide keys and values. The decoder retrieves a weighted combination of encoder value vectors according to these attention weights.

Local transformation. Once these pieces of information are gathered, the model must transform them into a representation suitable for prediction. The Transformer performs this step using a position-wise feed-forward network (FFN), a nonlinear transformation applied independently to each token representation.

Together these components replace the recurrent update. Rather than computing a single evolving state s_t, the Transformer repeatedly refines the representations of all tokens in the sequence.

Stacking Transformer Layers

A single decoder block performs one round of communication (attention) followed by a local nonlinear transformation (FFN). One such update is rarely sufficient. As in earlier sequence models, these blocks are stacked so that representations can be progressively refined:

$$X^{(0)} \to X^{(1)} \to X^{(2)} \to \cdots \to X^{(L)}.$$

Each layer receives the representation produced by the previous layer and computes a new transformation of it.

Deep stacks of nonlinear transformations are difficult to train. Gradients must propagate through many layers, and the signal can easily degrade.

Recurrent networks partially avoided this problem because their gating mechanisms allowed linear paths through the computation, helping gradients flow across time.

When designing very deep feed-forward networks, researchers encountered the same difficulty.

Residual networks (ResNets) addressed this problem by adding a direct skip connection across each block. Instead of learning a complete transformation $F(x)$, each block learns only a residual correction:

$$\text{output} = x + F(x).$$

The skip connection ensures that gradients can flow directly through the identity path while the block learns only what needs to change.

Transformers adopt the same structure. Each decoder block predicts a correction to the current representation rather than replacing it:

$$X^{(l+1)} = X^{(l)} + \Delta^{(l)}.$$

Here $\Delta^{(l)}$ represents the combined contribution of attention and the feed-forward network in layer l.

Replace versus accumulate Recurrent decoders update their state by replacement:

$$s_t \leftarrow f(s_{t-1}, y_{t-1}, c_t).$$

The previous state is consumed and replaced by the new one.

Transformers instead accumulate corrections:

$$X^{(l+1)} = X^{(l)} + \Delta^{(l)}.$$

Computation therefore moves from the time axis to the depth axis. Sequential overwriting is replaced by parallel accumulation.

Accumulation Across Depth

Transformer layers are stacked, producing a sequence of representations

$$X^{(0)} \to X^{(1)} \to X^{(2)} \to \cdots \to X^{(L)}.$$

Each layer contributes a correction to the representation through its residual update:

$$X^{(l+1)} = X^{(l)} + \Delta^{(l)}.$$

Unrolling the residual updates across all L layers makes the accumulation explicit:

$$X^{(L)} = X^{(0)} + \sum_{l=1}^{L} \big(\Delta^{(l)}_{\text{att}} + \Delta^{(l)}_{\text{ffn}}\big).$$

The representation produced by the network is therefore the initial embedding plus every correction contributed by each layer. Nothing is overwritten; every layer adds information on top of what previous layers have already constructed.

Consider *"The black cat sleeps on the rug."* In an early layer, self-attention resolves local relationships: *sleeps* attends to *cat* to pull in its subject; *black* attaches to *cat* as its modifier. By the time layer 1 finishes, the representation of *sleeps* already encodes something about its subject. Layer 2 inherits that result and can ask a higher-order question — not *what is the subject?* (already answered) but *what does this predicate structure imply for what comes next?* Each layer builds on all the layers before it, because nothing was discarded.

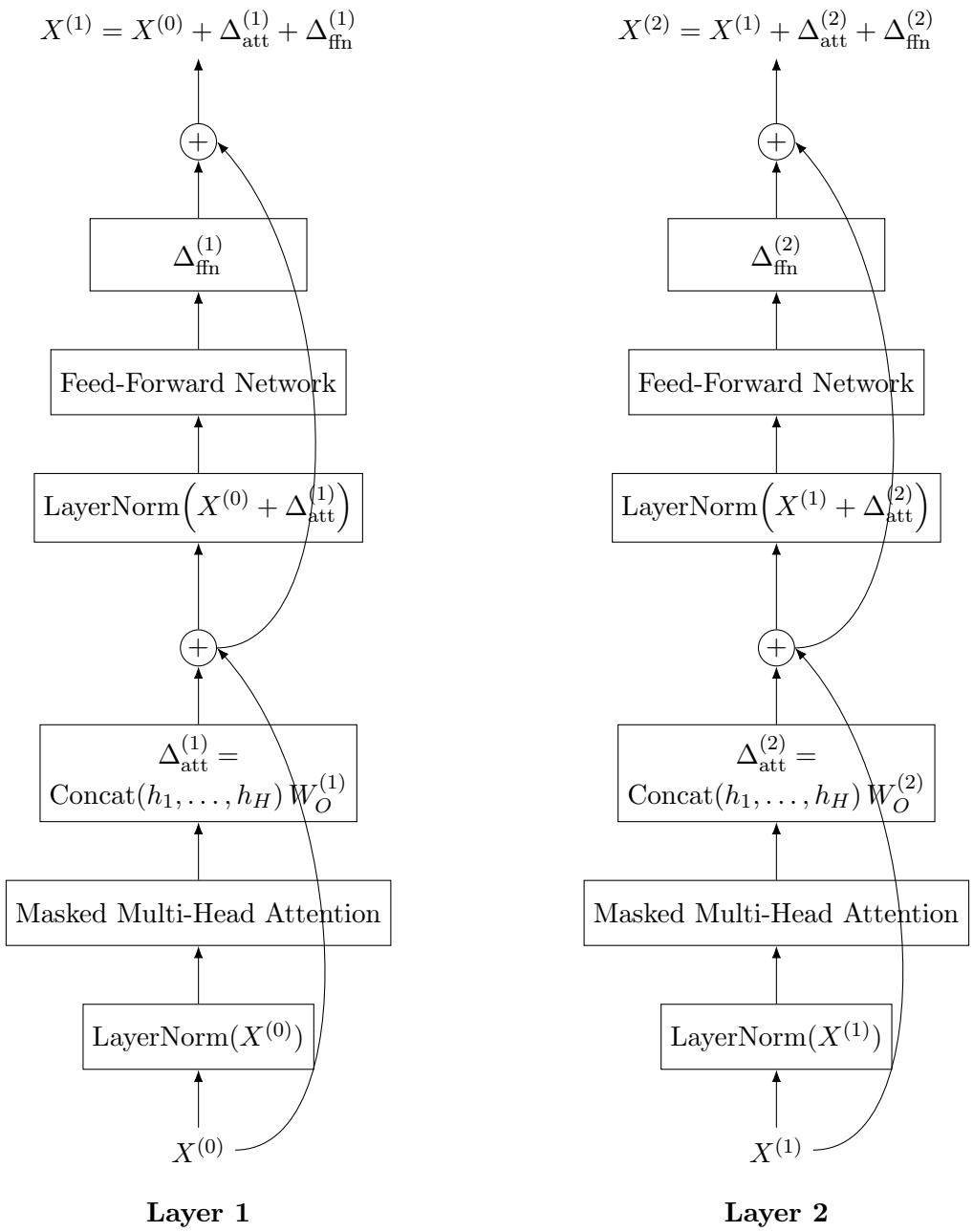

Figure 1.12: Two decoder layers of a pre-norm Transformer. Each layer applies masked self-attention then FFN, with LayerNorm before each block and residual connections around each. The skip connections carry the running representation forward; each block contributes only a delta. After L layers the final representations are projected into vocabulary space.

From Representations to Predictions

After L layers, each row $x_i^{(L)}$ of $X^{(L)}$ is a rich contextual representation of token i — the initial embedding plus every correction contributed by the stacked Transformer layers. This

vector is not yet a prediction. It is a summary of everything the model knows about position i in context. The question is how to turn that summary into a probability distribution over the next word.

The idea is simple: measure how compatible this representation is with every word in the vocabulary. Each word has an embedding vector in the matrix E, which represents the meaning of that word in the model's embedding space. Training encourages the representation $x_i^{(L)}$ to align with the embedding of the word that should come next, while remaining dissimilar to the embeddings of other words.

This compatibility is computed by projecting the representation into vocabulary space:

$$z_i = W_o\, x_i^{(L)} + b_o,$$

where $z_i \in \mathbb{R}^{|\mathcal{V}|}$ assigns a score (logit) to every word in the vocabulary.

In practice, the output projection W_o is often tied to the input embedding matrix E. The same vectors that encode what words mean when they enter the model are reused to score which word should come next.

These scores are converted into a probability distribution using softmax:

$$p(w \mid x_{<i}) = \frac{\exp(z_i[w])}{\sum_{w'} \exp(z_i[w'])}.$$

Every word receives a probability, and the probabilities sum to one. The word with the highest probability is the model's most confident prediction for the next token at position i.

Because

$$x_i^{(L)} = x_i^{(0)} + \sum_{l=1}^{L} \Delta^{(l)},$$

the final prediction reflects the accumulated contributions of all layers. The logits are not produced by the final layer alone; they result from every round of attention and transformation that refined the representation along the way.

Training and Inference

Training objective. The model is trained to predict each word in a sequence given the words that came before it. Given a sentence $(x_1, x_2, \ldots, x_N)$, the model is asked to assign high probability to x_2 given x_1, high probability to x_3 given x_1, x_2, and so on for every position. This left-to-right factorization is the autoregressive property:

$$p_\theta(x_1, \ldots, x_N) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}).$$

The better the model's predictions, the higher this probability. Training maximizes it — equivalently, it minimizes the cross-entropy loss, which penalizes the model for assigning low probability to the word that actually appeared:

$$\mathcal{J}(\theta) = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

Why training can be fully parallel. In the recurrent decoder, training was sequential: computing the hidden state at step t required the hidden state at step $t-1$ to already be ready. This chain of dependencies serialised the entire forward pass — every position waited for the one before it.

The Transformer breaks this dependency entirely. During training, the full sequence $(x_1, \ldots, x_N)$ is fed to the model at once. The causal mask does the work that the sequential chain used to do: it ensures that when computing the representation at position i, attention cannot see any token at position $j > i$. So $x_i^{(L)}$ depends only on $(x_1, \ldots, x_{i-1})$ — exactly the prefix it should condition on — but this guarantee comes from a mask applied in parallel, not from running the model step by step.

The result is that $X^{(L)}$ contains all N prefix-conditioned representations at once. Every conditional distribution

$$p_\theta(x_1 \mid \varnothing),\ p_\theta(x_2 \mid x_1),\ \ldots,\ p_\theta(x_N \mid x_{<N})$$

is computed in a single forward pass. The loss over the entire sequence is computed and gradients are propagated in one shot. This is what makes training on sequences of thousands of tokens tractable.

Autoregressive inference. At inference time, the situation is different. There is no full sequence to feed in — the model is generating one that does not yet exist. Only a prefix $(x_1, \ldots, x_k)$ is available.

The model processes this prefix, and the representation at the final position encodes everything the model knows about what should come next. It computes $p_\theta(x_{k+1} \mid x_{\leq k})$ and selects a token — by taking the most likely word, or by sampling. That token is appended to the prefix, and the whole process runs again for position $k+2$.

Unlike training, this is inherently sequential. Each new token depends on the one just generated, so the model must run once per token. The causal mask is still there — it is just that every "future" position is genuinely unknown rather than deliberately hidden.

The full picture. Every component in the Transformer was forced by a concrete problem. Token embeddings convert discrete symbols into vectors. Positional encodings restore the notion of order that recurrence previously provided. Masked self-attention replaces the recurrent target history without requiring sequential processing. Cross-attention retrieves information from the source sequence. The feed-forward network replaces the nonlinear integration once performed by the recurrent update f. Residual connections, borrowed from ResNet, allow these transformations to accumulate across depth without destroying earlier computations. Each component is the answer to a question that arose when the recurrent chain was removed.

The architecture therefore accumulates corrections across depth rather than overwriting state across time. Initial token embeddings enter the first layer and emerge from layer L as rich contextual representations after repeated rounds of communication (attention) and transformation (FFN). Next-token prediction is then a simple linear read-off from that representation stream.

This process of continually transforming representations is reflected in the name **Transformer**. Somewhere in the model's d_{model}-dimensional representation space lie directions

and regions corresponding to concepts expressed in language. A token embedding provides only a context-independent starting point in this space. Words like *bank* begin with the same vector regardless of whether the sentence refers to finance or a river.

The role of the Transformer is to transform this initial vector into a contextual representation that reflects the meaning of the word in its specific sentence. Through repeated interactions with surrounding tokens, the representation moves to a region of space that encodes the model's interpretation of the sentence and aligns with directions corresponding to words that are likely to follow.

In this sense, the Transformer does not store meanings for words directly. Instead, it computes meaning dynamically by moving token representations through a high-dimensional space until they reach positions that best explain the sentence and predict what comes next.

Worked Example: A Complete Pre-Norm Transformer Block

The preceding sections described the Transformer's components — LayerNorm, multi-head self-attention, residual connections, and the feed-forward network — at the level of equations and diagrams. This section works through a single Pre-Norm block numerically from raw token embeddings to the final output, tracking every matrix computation with concrete numbers.

The Pre-Norm block in one equation. LayerNorm is applied *before* each sub-layer, not after:

$$X_{\text{attn}} = X + \text{MHA}\big(\text{LN}(X)\big), \qquad X_{\text{out}} = X_{\text{attn}} + \text{FFN}\big(\text{LN}(X_{\text{attn}})\big).$$

The raw input X flows unchanged through both skip connections; only the normalized branch enters each sub-layer.

Toy configuration.

Symbol	Meaning	Value
n	sequence length (tokens)	3
d_{model}	embedding dimension	4
h	number of attention heads	2
d_{head}	per-head dimension ($= d_{\text{model}}/h$)	2

Step 1 — Raw input.

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 4}.$$

Rows are tokens w_1, w_2, w_3. X is the *residual stream* — the block only ever adds increments to it; it is never overwritten.

Step 2 — LayerNorm before attention. Applied independently to each row: $\text{LN}(x) = (x-\mu)/\sigma$ where μ, σ are the row mean and standard deviation.

w_1**:** $\mu = 0.5,\ \sigma = 0.5 \ \Rightarrow\ \tilde{x}_1 = [1, -1, 1, -1]$.

w_2**:** $\mu = 0.5,\ \sigma = 0.5 \ \Rightarrow\ \tilde{x}_2 = [-1, 1, 1, -1]$.

w_3**:** $\mu = 0.75,\ \sigma \approx 0.433 \ \Rightarrow\ \tilde{x}_3 \approx [0.58, 0.58, -1.73, 0.58]$.

$$\tilde{X} = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 0.58 & 0.58 & -1.73 & 0.58 \end{bmatrix}.$$

Projections use $\tilde{X}$, skip connections use X. All query/key/value matrices are computed from $\tilde{X}$. The raw X appears *only* in the residual additions below.

Step 3 — Multi-head self-attention. Each head h projects $\tilde{X}$ into 2-dimensional Q/K/V spaces via $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{4\times 2}$.
Head 1 — projections:

$$W_Q{}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K{}^{(1)} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_V{}^{(1)} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

$$Q^{(1)} = \begin{bmatrix} 2 & -2 \\ 0 & 0 \\ -1.15 & 1.15 \end{bmatrix}, \quad K^{(1)} = \begin{bmatrix} 2 & -1 \\ 0 & 1 \\ -1.15 & -0.58 \end{bmatrix}, \quad V^{(1)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1.16 & -1.15 \end{bmatrix}.$$

Scores, causal mask ($-\infty$ for $j > i$), and softmax:

$$S^{(1)}_{\text{masked}} = \begin{bmatrix} 6 & -\infty & -\infty \\ 0 & 0 & -\infty \\ -3.45 & 1.15 & 0.66 \end{bmatrix}, \quad A^{(1)} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0.006 & 0.61 & 0.38 \end{bmatrix}.$$

Head 2 — projections:

$$W_Q{}^{(2)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_K{}^{(2)} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad W_V{}^{(2)} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

$$S^{(2)}_{\text{masked}} = \begin{bmatrix} 0 & -\infty & -\infty \\ -2 & 0 & -\infty \\ 1.15 & -1.15 & -1.33 \end{bmatrix}, \quad A^{(2)} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.12 & 0.88 & 0 \\ 0.84 & 0.10 & 0.06 \end{bmatrix}.$$

Two heads, two patterns. Head 1 has w_3 attending mostly to w_2 (0.61); Head 2 has w_3 attending mostly to w_1 (0.84). Each head independently extracts a different relational signal from the same sequence — the core motivation for multi-head attention.

Step 4 — Contextual outputs and concatenation. $C^{(h)} = A^{(h)}V^{(h)} \in \mathbb{R}^{3\times 2}$:

$$C^{(1)} \approx \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0.44 & -0.44 \end{bmatrix}, \quad C^{(2)} \approx \begin{bmatrix} 2.00 & -1.00 \\ 0.24 & -0.12 \\ 1.61 & -0.77 \end{bmatrix}.$$

Concatenate heads and apply output projection $W_O = I$:

$$\Delta_{\text{attn}} = [C^{(1)} \mid C^{(2)}] = \begin{bmatrix} 0 & 0 & 2.00 & -1.00 \\ 0 & 0 & 0.24 & -0.12 \\ 0.44 & -0.44 & 1.61 & -0.77 \end{bmatrix} \in \mathbb{R}^{3\times 4}.$$

Step 5 — First residual connection.

$$\begin{aligned} X_{\text{attn}} &= X + \Delta_{\text{attn}} \\ &= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 2.00 & -1.00 \\ 0 & 0 & 0.24 & -0.12 \\ 0.44 & -0.44 & 1.61 & -0.77 \end{bmatrix} \\ &\approx \begin{bmatrix} 1 & 0 & 3.00 & -1.00 \\ 0 & 1 & 1.24 & -0.12 \\ 1.44 & 0.56 & 1.61 & 0.23 \end{bmatrix} \end{aligned}$$

Step 6 — LayerNorm before FFN.

$$X_{\text{FFN-norm}} = \text{LN}(X_{\text{attn}}) \approx \begin{bmatrix} 0.39 & -0.65 & 1.43 & -1.17 \\ -0.94 & 0.89 & 1.11 & -1.05 \\ 1.68 & -0.15 & -0.69 & -0.84 \end{bmatrix}.$$

Step 7 — Feed-forward network. The FFN $\text{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2$ is applied token-wise to $X_{\text{FFN-norm}}$:

$$\Delta_{\text{FFN}} \approx \begin{bmatrix} 0.22 & -0.08 & 0.31 & -0.06 \\ -0.18 & 0.27 & 0.12 & -0.21 \\ 0.36 & -0.04 & -0.11 & 0.14 \end{bmatrix}.$$

Step 8 — Second residual connection.

$$X_{\text{out}} = X_{\text{attn}} + \Delta_{\text{FFN}} \approx \begin{bmatrix} 1.22 & -0.08 & 3.31 & -1.06 \\ -0.18 & 1.27 & 1.36 & -0.33 \\ 1.80 & 0.52 & 1.50 & 0.37 \end{bmatrix}.$$

The residual stream view. The block is three matrices added together:

$$\boxed{X_{\text{out}} = X + \Delta_{\text{attn}} + \Delta_{\text{FFN}}.}$$

$$\underbrace{\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}}_{X} + \underbrace{\begin{bmatrix} 0 & 0 & 2.00 & -1.00 \\ 0 & 0 & 0.24 & -0.12 \\ 0.44 & -0.44 & 1.61 & -0.77 \end{bmatrix}}_{\Delta_{\text{attn}}} + \underbrace{\begin{bmatrix} 0.22 & -0.08 & 0.31 & -0.06 \\ -0.18 & 0.27 & 0.12 & -0.21 \\ 0.36 & -0.04 & -0.11 & 0.14 \end{bmatrix}}_{\Delta_{\text{FFN}}} \approx X_{\text{out}}.$$

> **Key takeaway.** A transformer block never *overwrites* representations — it *accumulates* them. Attention writes a context-aware increment Δ_{attn}; the FFN writes a local nonlinear increment Δ_{FFN}. Both gradients flow unimpeded backward through the skip connections, bypassing every nonlinearity — this is why deep transformers are trainable despite their depth.

Encoder–Decoder Transformer for Sequence Transduction

The decoder-only architecture described above is designed for language modelling: given a sequence of tokens, predict the next one. A different class of tasks requires **sequence transduction** — mapping one sequence to another, such as translating English to French. For this, the Transformer adopts an encoder–decoder structure.

The Encoder. The encoder processes the entire source sequence at once. Because the full source is available before decoding begins, there is no need to restrict tokens from attending to future positions — the causal mask is removed. Each encoder layer applies unmasked self-attention followed by a feed-forward network:

$$\begin{aligned} X' &= X + \text{MHA}(\text{LayerNorm}(X)), \\ Y &= X' + \text{FFN}(\text{LayerNorm}(X')). \end{aligned}$$

After multiple encoder layers, the source sequence is represented as a matrix of contextual vectors:

$$H^{\text{enc}} \in \mathbb{R}^{N \times d_{\text{model}}},$$

where each row is a richly contextualized representation of a source token.

Cross-Attention. The decoder generates the target sequence autoregressively, one token at a time. In addition to masked self-attention over previously generated tokens, each decoder layer contains a **cross-attention** block that connects the decoder to the encoder output.

In cross-attention, queries come from the decoder and keys and values come from the encoder:

$$Q = X^{\text{dec}} W_Q, \qquad K = H^{\text{enc}} W_K, \qquad V = H^{\text{enc}} W_V.$$

Attention is then computed as:

$$\text{CrossAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

No causal mask is applied here — the decoder is free to attend to any encoder position. Each decoder token queries the full source representation and retrieves what is most relevant for predicting the next target token.

Decoder Layer Structure. Each encoder–decoder decoder layer now contains three sub-blocks:

$$\begin{aligned} X' &= X + \text{MaskedMHA}(\text{LayerNorm}(X)), \\ X'' &= X' + \text{CrossMHA}(\text{LayerNorm}(X'), H^{\text{enc}}), \\ Y &= X'' + \text{FFN}(\text{LayerNorm}(X'')). \end{aligned}$$

The masked self-attention maintains autoregressive generation over the target sequence, cross-attention retrieves relevant source context, and the FFN refines the representation locally.

Summary. In sequence transduction, the encoder builds a full contextual representation of the source without masking, while the decoder generates the target autoregressively, attending to both its own past outputs and the encoder representation through cross-attention. The layer-by-layer refinement described earlier applies to both the encoder and the decoder — the Transformer transforms both sequences into progressively richer representations until prediction becomes easy.

BERT 2018 , Developed by Google in 2018, it is a revolutionary machine learning framework for natural language processing (NLP) that uses a transformer-based, encoder-only architecture to understand the context of words in search queries and text.

Definition 1.16: GPT (Generative Pre-trained Transformer)

GPT is a decoder-only Transformer that uses causal masked self-attention and is trained autoregressively with the objective

$$\max_{\theta} \sum_{t} \log p_\theta(x_t \mid x_1, \ldots, x_{t-1}).$$

Advantages over RNNs:

- **Parallel training:** all positions are processed at once, removing the sequential bottleneck during training.
- **Direct long-range access:** any two positions interact through a single attention step, avoiding repeated transformations across time.
- **No compression bottleneck:** instead of storing history in a fixed-size hidden state, attention retrieves relevant information directly.
- **Scalability:** performance improves reliably with more data, parameters, and compute.

Generation remains sequential: token t must be produced before token $t + 1$.

The key conceptual shift from RNNs to attention is a move from *compression* to *retrieval*. An RNN must summarize the entire past into a single hidden state before knowing what future steps will need. Attention postpones this decision: each step directly retrieves whatever past information is most relevant.

Historical Evolution of Ideas Behind the Transformer

The Transformer architecture did not emerge in isolation. It is the result of several lines of research that progressively shaped its components.

- **Recurrent Neural Networks (1990s)** Early sequence models such as RNNs introduced the idea of processing tokens sequentially while accumulating history in a hidden state. This established the need for contextual representations.

- **LSTM and GRU (1997–2014)** Gated recurrent models addressed long-range dependencies and enabled deep stacked sequence models. The practice of stacking multiple layers later reappears in Transformer architectures.
- **Encoder–Decoder Seq2Seq Models (2014)** Neural machine translation introduced the encoder–decoder paradigm, where an input sequence is encoded into representations used by a decoder to generate outputs. The original Transformer preserves this high-level structure.
- **Early Attention Mechanisms (2014–2015)** Attention replaced the fixed-size context vector with dynamically computed weighted combinations of encoder states. This removed the bottleneck of compressing all information into a single vector and directly inspired self-attention.
- **MADE and Autoregressive Masking (2015)** Masked feed-forward networks demonstrated that autoregressive generation can be enforced using structural masking. Transformer decoders adopt the same principle through causal masking in self-attention.
- **Residual Networks (2015)** Skip connections enabled stable training of deep networks. Transformers apply residual connections around both attention and feed-forward blocks.
- **Layer Normalization (2016)** Layer normalization stabilized deep sequence models and became a standard component of Transformer layers.
- **Feed-Forward Attention Models (2016)** Work on attention-based models without recurrence showed that sequence modeling could be parallelized, motivating fully attention-based architectures.
- **Convolutional Seq2Seq Models (2017)** Fully parallel sequence models using convolution demonstrated that recurrence was not strictly necessary, paving the way for the Transformer's non-recurrent design.

Summary. The Transformer can be viewed as a synthesis of earlier ideas: stacked sequence modeling from RNNs, dynamic context via attention, autoregressive masking from feed-forward autoregressive models, residual learning from deep vision networks, and fully parallel computation from convolutional and attention-based sequence models.

1.6.5 Summary: Variable-Length Models

From recurrence to attention. LSTMs and GRUs solved the vanishing gradient problem but introduced a new one: the entire history of a sequence must be compressed into a fixed-size hidden state. This bottleneck is most visible in encoder–decoder models for machine translation, where performance degrades on long sentences because one vector cannot carry everything the decoder needs. Attention was introduced to alleviate this bottleneck: the decoder looks back at all encoder hidden states and forms a weighted sum, placing more weight on positions relevant to the current step. The weights are learned and input-dependent, giving the model soft control over what to retrieve — the same principle that made the LSTM cell state effective.

This idea turns out to be more than a patch. Recurrence was never the goal; it was simply a mechanism for modeling dependencies between positions by passing information forward step by step. Attention provides a different solution: each position can directly access any other, modeling dependencies in a single step rather than propagating them across many. This also

removes the sequential constraint inherent in recurrence, allowing all positions to interact in parallel and making training much faster on long sequences.

One issue remains: causal order. During generation, the prediction at position i must depend only on earlier positions. Recurrence enforces this naturally, but attention requires it to be imposed explicitly through masking (similar to the autoregressive masking used in MADE), so that information cannot flow from future to past positions.

The Transformer follows directly from these ideas: replace recurrence with attention as the primary mechanism for modeling dependencies, enforce causal structure through masking, and build the model around parallel, weighted aggregation of information combined with nonlinear transformations.